Advanced Vision and Memory for traditional World Models

Author:
Lavanya
Chinthakayala
303066

Supervisors:
Prof. Dr. Dr. Lars Schmidt-thieme
Daniela Thyssens

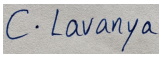
12th November 2021

Thesis submitted for
Master of Science in Data Analytics

Wirtschaftsinformatik und Maschinelles Lernen
Stiftung Universität Hildesheim
Universitatsplätz 1, 31141 Hildesheim

Statement as to the sole authorship of the thesis: Advanced Vision and Memory for traditional World Models

I hereby certify that the master's thesis named above was solely written by me and that no assistance was used other than that cited. The passages in this thesis that were taken verbatim or with the same sense as that of other works have been identified in each individual case by the citation of the source or the origin, including the secondary sources used. This also applies for drawings, sketches, illustrations as well as internet sources and other collections of electronic texts or data, etc. The submitted thesis has not been previously used for the fulfilment of a degree requirements and has not been published in English or any other language. I am aware of the fact that false declarations will be treated as fraud.



12-11-2021, Hildesheim

Abstract

The task of training the Reinforcement Learning (RL) model in real-world environments is challenging because it is expensive in terms of cost as well as time. This induces the use of simulators, which play a key role in training Deep RL algorithms by imitating the real-world environment. However, simulating real-world environments is often cumbersome and constitutes another challenging task. To mitigate this problem, a brilliant approach is explained in the paper "World Models" where an agent can be trained in its own simulated environment and then project the learned policy into the real-world environment. Their work provides numerous advantages like controlling the environment using low dimensional, internal representations and generating simulated data for training by learning the transition dynamics of the environment. The main motivation of this thesis is to improve the vision and memory components of traditional world models in order to better learn the spatial and temporal representation of the real-world environment. This research mainly focuses on overcoming the limitations of the emplaced vision (VAE) network by using advanced generative neural networks like InfoVAE, β-VAE. The improvement of the vision (V) model leads to the implicit improvement of the memory (M) model because the memory (M) model depends on the output of the vision (V) model. Finally, this thesis work aims to experiment with different deep generative neural networks in order to learn the high-quality spatial and temporal representation of the real-world environment, which in turn generates the most informative simulation data for training.

Contents

List of Tables

List of Figures

List of Abbreviations

AdaGrad	Adaptive Gradient Algorithm
ANN	Artificial Neural Network
BPTT	Backpropagation Through Time
CMA-ES	Covariance-Matrix Adaptation Evolution Strategy
CNN	Convolutional Neural Network
DDP	Differential Dynamic Programming
DNN	Deep Neural Network
DP	Dynamic Programming
ELBO	Evidence Lower Bound
EM	Expectation Maximization
ES	Evolution Strategies
GRU	Gated Recurrent Unit
InfoVAE	Information Maximizing Variational AutoEncoder
KL	Kullback-Leibler
LSTM	Long Short-Term Memory
MC	Monte Carlo
MDN	Mixture Density Network
MDP	Markov Decision Process
MLP	Multi Layer Perceptron
MMD-VAE	Maximum Mean Discrepancy Variational AutoEncoder
MSE	Mean Squared Error
RL	Reinforcement Learning
RMSProp	Root Mean Squared Propagation
RNN	Recurrent Neural Network
SGD	Stochastic Gradient Descent
SLP	Single Layer Perceptron
TDL	Temporal Difference Learning
VAE	Variational AutoEncoder
VQ-VAE	Vector Quantized-Variational AutoEncoder

Chapter 1

Introduction

Deep Reinforcement learning has become one of the most exciting fields in robotics and artificial intelligence (Henderson et al., 2018a). The large amount of training data that is used to train DRL agents plays a major role in effectively learning a policy that leads to higher cumulative rewards. Both the training process and collection of training data for deep RL algorithms are computationally expensive. Researchers found that simulation data can be used to train deep RL agents to significantly increase the accuracy. Tremendous research is going on how to train an RL agent using simulation data. One of the latest research (Ha and Schmidhuber, 2018) proved that an agent can learn a policy using its generated simulated data. This thesis work focuses on improving the quality of generated simulation data from the "World Models" (Ha and Schmidhuber, 2018) by using different combinations of deep learning algorithms.

This chapter presents different sections where Section 1.1 explains the clear motivation behind this research, Section 1.2 introduces the research questions and Section 1.3 provides the outline of the thesis work.

1.1 Motivation

With the advent of Deep RL, there has been tremendous research going on how the various deep neural networks can be incorporated into reinforcement learning (RL) to maximize the cumulative reward. The task of training these RL algorithms in real-world environments is challenging because it is expensive in terms of cost as well as time. There comes simulators that play a key role in training the Deep RL algorithms by imitating the real-world environment (Schatzmann et al., 2006). Simulating the real-world environment is again another challenging task (Liang et al., 2018a).

To mitigate the above problem, a brilliant approach is explained in the paper "World Models (Ha and Schmidhuber, 2018)" where a reinforcement learning agent can be trained in its own simulated environment and then project the learned policy into the real-world environment. The architecture of world models constitutes three different components called vision (V), memory (M), and controller (C). The vision (V) component gives the compressed representation of the environment whereas the memory (M) component predicts future frames by taking the compressed representation as input from the vision (V) component. In the end, the controller (C) component takes the compressed representation from the vision (V) component and predicted future frame from the memory (M) component then decides on which action to take that constitutes the higher reward. Their work provides numerous advantages like controlling the environment using low dimensional, internal representations and also generating simulated data for training by learning the transition dynamics of the environment.

The main motivation of this thesis is to improve the vision (VAE) and memory (LSTM) networks of traditional world models (Ha and Schmidhuber, 2018) to better learn the spatial and temporal representation of the real-world environment. This research mainly focuses on overcoming the limitations of the existing vision (VAE) network by using advanced generative neural networks like InfoVAE, beta-VAE to learn the important features effectively. Finally, this thesis aims to experiment with different deep neural networks to learn the high-quality spatial and temporal representation of real-world environments which in turn generates better simulation data for training. The better simulation data generated from this thesis work should help the RL agent to learn the optimal policy which maximizes the cumulative reward.

1.2 Research Questions

To improve the overall performance of traditional world models, this thesis aims to answer the following research questions.

1. Does this research help to improve the performance of the existing traditional world model?
2. How good is the advanced vision model compared to the existing vision model?
3. How are the results of the improved model evaluated?

1.3 Thesis Outline

The organization of this thesis report is as follows.

- Chapter 2 reviews and summarizes the existing research work in regards to world models with different approaches. Also, this section reviews the research works that are related to existing vision and memory models in the baseline approach.

- Chapter 3 introduces the fundamental concepts of deep learning and reinforcement learning that are required to better understand this thesis work.

- Chapter 4 describes the baseline and improved methodologies that are used to evaluate the performance of the vision and memory model.

- Chapter 5 introduces the dataset that is used in this research along with its preprocessing techniques. This chapter also presents the experimental results from both the baseline and proposed models. These results are used to discuss and analyze the insights gained by this research.

- Chapter 6 concludes the thesis by summarizing the key contributions and provides suggestions for future research directions.

Chapter 2

Literature Review

This chapter provides an overview of literature related to world models. It also reviews the existing deep learning approaches present in the literature which helps to improve the traditional world models. Section 2.1.1 reviews the relevant work related to the improvement of the vision (V) model. Section 2.1.2 discusses the literature related to the improvement of memory (M) model.

2.1 Related Work

World Models: This paper by Ha and Schmidhuber (2018) demonstrates how the generative neural network models can be used to train the RL agent in its simulated environment. The main three components in their work are vision, memory, and controller. Vision gives the latent representation of high dimensional data, memory predicts the next sequence of images and the controller decides which action to take by an agent to get the better reward. VAE (Kingma and Welling, 2013) has been used for vision network, MDN-RNN (LSTM)((Bishop, 1994) has been used for memory network, Covariance-Matrix Adaptation Evolution Strategy (CMA-ES) has been used for optimizing the controller. Though this approach has numerous advantages it has a few limitations with the vision model. Vision (VAE) network encodes the non-relevant task data. This paper is considered as a baseline architecture for this research and the detailed architecture will be explained in Chapter 5. The above-stated limitation is addressed in this thesis work.

Dream to Control: Inspired by the world models, Hafner et al. (2019) proposed a model called 'Dreamer' which can efficiently learn behaviors from the latent imagination of a trained world model. After the world models learn from high-dimensional inputs they can be trained on their own simulated

data (Ha and Schmidhuber, 2018). Thus the authors Hafner et al. (2019) leveraged the trained world model to derive behaviors by imagining in the compact state space.

World Models Without Forward Prediction: In this work, authors Freeman et al. (2019) claimed that there is no need for world models to perform explicit forward prediction and believed that, if it is useful to learn the environment then the agent will predict on its own. Thus they constrained the input observation to the agent at each time step. They concluded that if the agent is trained without forward prediction then it can learn the key skills necessary to achieve good performance in a given environment.

Mastering Atari with Discrete World Models: Following the Dreamer, (Freeman et al., 2019) authors Hafner et al. (2020) proposed a 'DreamerV2' with discretization of latent space. Instead of using Gaussian latent variables, they train the world model using discrete latent space.

Smaller World Models for Reinforcement Learning: Robine et al. (2020) proposed a variation of the world model with a significantly smaller number of samples. They used vector quantized-variational autoencoder (VQ-VAE) as their vision model and convolutional lstm as memory model along with PPO optimization technique for a controller. They demonstrated comparable results in various Atari environments.

Deep Neuroevolution of Recurrent and Discrete World Models: As the architecture of the world model constitutes multiple components like vision, memory, and decision making controller, Risi and Stanley (2019) introduced a genetic algorithm which can perform all three components using an end to end approach. Also, they achieved comparable performance to original world models using car racing task.

Attention world models: Chadha and Bablani (2018) proposed another variation of world models which uses gating mechanisms in the vision model to learn descriptive factorized latent feature vectors and also used attention mechanism on top of memory model to learn the focused spatial representation.

Neuroevolution of self-interpretable agents: Motivated by self-attention techniques Tang et al. (2020) studied the properties of artificial agents which see the world through the lens of a self-attention bottleneck. By training with the self-attention architectures their agents can generalize to environments

where task-irrelevant elements are modified while existing methods fail.

VMAV-C: VMAV-C which is proposed by Liang et al. (2018b) refers to a combination of Variational Auto-Encoder (VAE), Mixture Density Network Recurrent Neural Network (MDN-RNN), Attention-based Value Function (AVF), and Controller Model. In this research by Liang et al. (2018b) they incorporated the attention mechanism in the state value function and used the Proximal Policy Optimization technique to optimize the controller. Even though they used the world models architecture as a baseline which looks very simple, their proposed architecture seems very complex.

2.1.1 Related work for vision model

Information Maximizing Variational Autoencoders (InfoVAE): Learning the distribution of input data to generate meaningful results from the encoded/latent distribution has gained a lot of attention and has been immensely useful in data generation for training machine learning algorithms. One of the most popular deep generative neural network models includes Variational Auto-Encoders. But it has the limitation on interpreting the latent representation of data. To mitigate this issue, a powerful technique called InfoVAE has been demonstrated in this paper by Zhao et al. (2017) which can make use of the latent features effectively to improve the quality of variational posterior. The goal of InfoVAE is to do the representation learning by encouraging large mutual information between latent space (Z) and input (X) by adding a regularizer of maximum mean discrepancy (MMD)(Zhao et al., 2017).

Understanding disentangling in β-VAE: The study of disentangled representation in VAE has gained a lot of attention. β-VAE (Higgins et al., 2016) is a slight modification to the traditional VAE which allows learning the disentangled latent factors effectively. Burgess et al. (2018) introduced a slight modification to the training process of β-VAE which increases the latent information capacity without compromising on reconstruction accuracy.

Variations in Variational Autoencoders - A Comparative Evaluation: The process of generating new meaningful data from a learned distribution of data has led to tremendous research. One such well-known deep generative model is VAE (Kingma and Welling, 2013). This paper by Wei et al. (2020) provides a comprehensive comparison between different variations of VAE including the advantages and disadvantages of each variation.

2.1.2 Related work for memory model

LSTM-MDN-ATTN: Park et al. (2019) introduced an LSTM-MDN-ATTN architecture to predict the medical time series data. LSTM-MDN model helps to predict the future value by approximating the distribution of target data. Attention mechanism on top of this model is used to better focus on the distribution which is related to target data.

Temporal Convolutional Networks (TCN) for sequence modeling: Sequence modeling plays a vital role because most of the existing data is sequential in nature. This technique helps to predict the next word, audio, video, or image based on the previous inputs. In general, recurrent neural networks such as Long Short Term Memory (LSTM) and Gated Recurrent Unit (GRU) are used for sequence modeling tasks (Bai et al., 2018). When the input sequences are very long these RNNs require a lot of memory to store results of cell gates and suffer from forgetting issues. To overcome this limitation, a powerful technique called Temporal Convolutional Network (TCN) (Bai et al., 2018) is introduced to have very long history sizes thus outperforming the canonical recurrent neural networks. TCN uses dilated causal convolutions and residual blocks to make it more suitable for sequential modeling tasks. In addition to longer effective memory sizes, TCN also has various advantages like parallelism, flexible receptive field size, etc.,

Chapter 3

Theoretical Background

This chapter reviews the background concepts related to deep learning and reinforcement learning. Section 3.3 describes the fundamentals of neural networks along with the optimization and regularization techniques. Section 3.5 gives the introduction of deep generative models. Section 3.7 explains the concepts like mixture density networks and Section 3.8 discusses the importance of evolution strategies in RL which helps to better understand the baseline architecture.

3.1 Artificial Intelligence

Artificial Intelligence (AI) is the science of using computer programs to make intelligent machines that can solve real-world problems with human-level intelligence (McCarthy, 2007). In other words, AI helps to simulate the decision-making and problem-solving capabilities of the human mind. AI is a broad domain to research and has been incorporated into various applications like automation, natural language processing, robotics, self-driving, and many more.

3.2 Machine Learning

Machine learning is the sub-domain of artificial intelligence. It enables a machine to learn on its own through experience and given data until the desired outcome is obtained. These machine learning algorithms help in predicting future outcomes with better accuracy. The following are the types of machine learning algorithms.

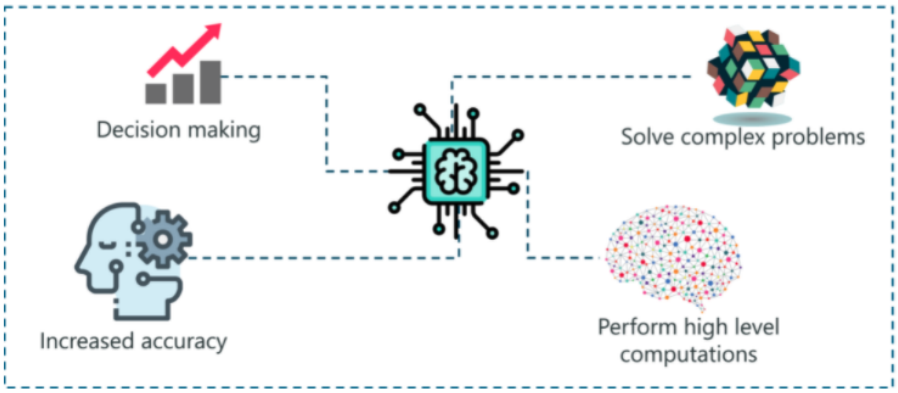


Figure 3.1: Artificial Intelligence
(Lateef, 2021)

3.2.1 Supervised learning

Supervised Learning uses data that is already labeled by humans for training the machine learning algorithm. Then the unknown data is given to the trained model to predict the desired outcome. Examples include linear regression, decision trees, etc.,

3.2.2 Unsupervised Learning

Unsupervised Learning uses unlabeled data for training. Here the machine learning algorithm tries to learn the patterns and underlying structure of the input data. Examples include K-means clustering, Principal component analysis.

3.2.3 Reinforcement Learning (RL)

Reinforcement Learning agent learns by interacting with the environment by trial and error method to maximize the cumulative reward (Henderson et al., 2018b). The agent takes an action to get the reward from the environment and adjusts its policy based on the obtained reward. The agent represents the machine learning model which is trained to learn to make the sequence of decisions.

3.3 Deep Learning

Deep Learning is the sub-domain of Machine Learning (ML). It is inspired by the behavior of the human brain and uses multiple neural network lay-

ers to extract useful features from the large dataset. One of the important distinctions between machine learning and deep learning algorithms is that **feature engineering** is done manually in machine learning whereas deep learning algorithms can extract and select the required features automatically.

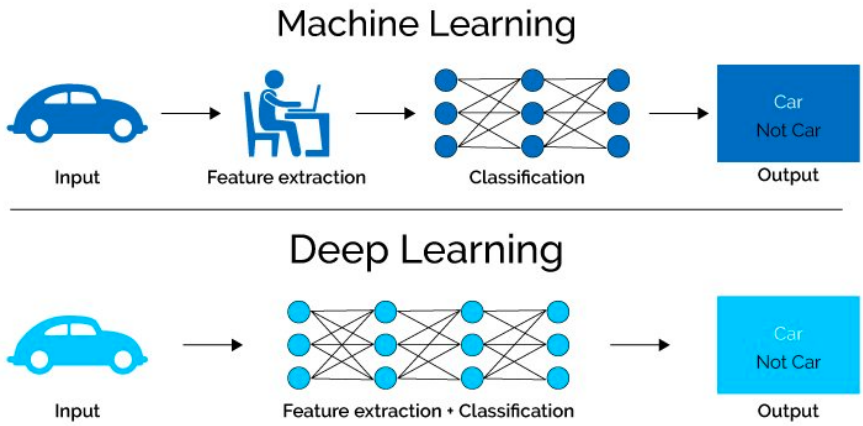


Figure 3.2: Comparison of Machine learning and deep learning (Pai, 2020)

3.3.1 Artificial Neuron

The smallest unit called the neuron is responsible to carry the information between the layers (Goodfellow et al., 2016).

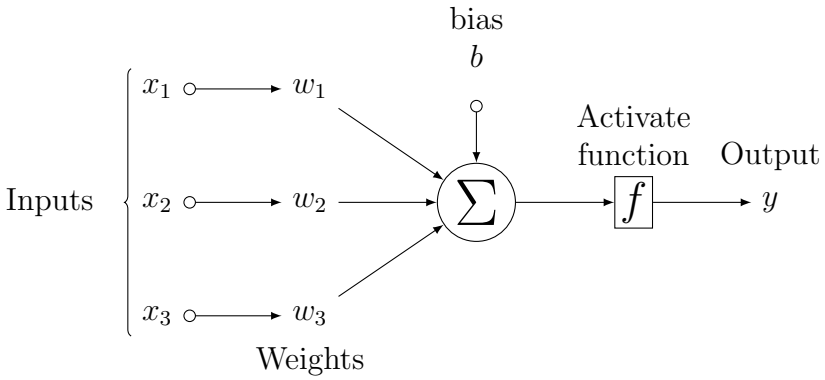


Figure 3.3: Artificial Neuron

A neuron accepts the inputs from the input vector $X = [x_1, x_2, \ldots x_n]$ and these inputs are multiplied with weight vector $W = [w_1, w_2, \ldots w_n]$ to

produce the output $y = (f(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b))$ where f denotes the activation function (Section 3.3.3) and b is the bias (constant).

3.3.2 Single-layer perceptron and Multi-layer perceptron

Single-layer perceptron (SLP) contains only an input layer and an output layer. It does not have any hidden layers in between. Neurons in the output layer take the weighted sum of all the inputs and give the output by applying the activation function (Section 3.3.3) on the weighted sum.
Multi-layer perceptron (MLP) consists of input, hidden, and output layers. It has one or more hidden layers in between and each hidden layer consists of multiple neurons called hidden units. The outputs from one layer are given as inputs to the next layer.

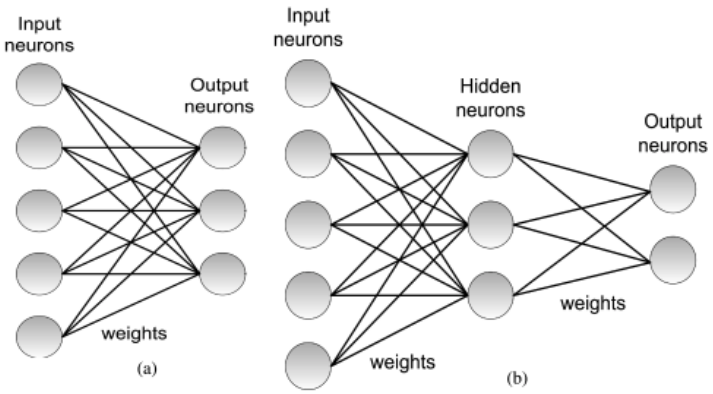


Figure 3.4: (a) Single layer perceptron (b) Multi layer perceptron (Camuñas-Mesa et al., 2019)

3.3.3 Activation function

The **Activation function** is responsible for introducing non-linearity in the network design. The choice of activation function is very important because it decides the type of output predictions. The neurons in the hidden layer and output layer take the weighted sum as input, then transforms the input into desired output by applying the activation function. The common types of activation functions are depicted in Figure 3.5

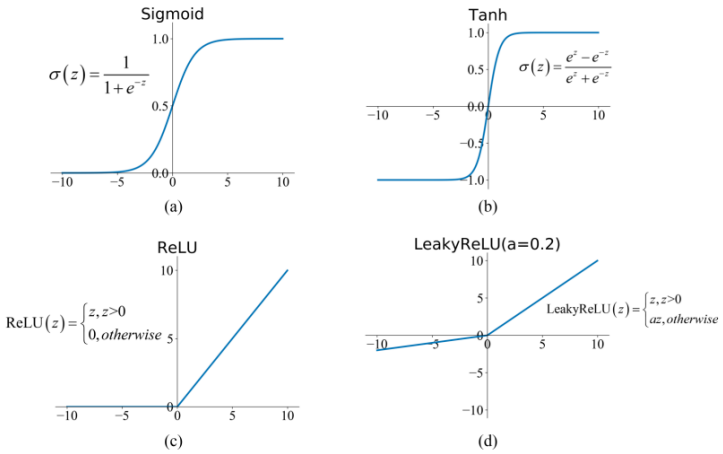


Figure 3.5: Types of Activation functions
(Feng et al., 2019)

3.3.4 Loss function

The **Loss function** is also known as cost function or error function which is used to evaluate the performance of the model. It tells how well a machine learning model is predicting the desired outputs. It is obtained by calculating the difference between the predicted output and the ground truth. The common loss functions include Mean Squared Error MSE and Cross-Entropy. MSE is used to solve the regression tasks and Cross-Entropy is used to solve the classification task (Goodfellow et al., 2016). The Loss functions for MSE and Cross-Entropy are as follows:

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 \tag{3.1}$$

$$L_{\text{cross-entropy}} = \frac{1}{N}\sum_{i=1}^{N} y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i) \tag{3.2}$$

where:

N = Total number of input samples
y_i = Ground truth
$\hat{y}_i$ = Predicted outcome

3.3.5 Backpropagation

The main goal of the machine learning model is to minimize the loss in order to generalize the data better. This goal is achieved by using the **backpropagation** technique which uses chain rule to train the machine learning model. It helps in adjusting the model parameters (weights and bias) to minimize the overall loss value. First, the model weights are initialized randomly, then the partial derivatives of loss function L are computed with respect to the parameters, and then these parameters are recursively updated until a global minimum is located. Figure 3.6 illustrates the backpropagation technique using the chain rule for a single neuron.

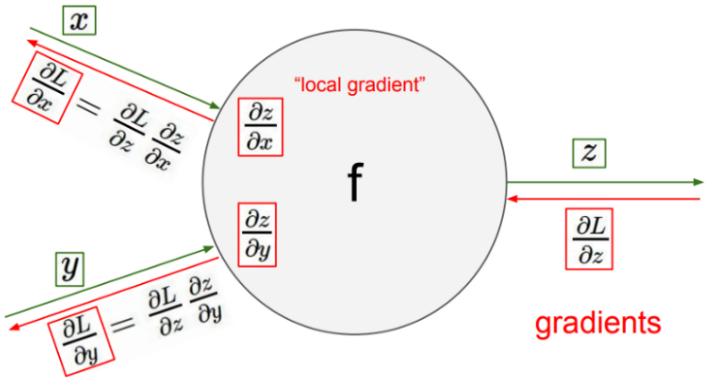


Figure 3.6: Backpropagation chain rule
(Sivamani, 2019)

3.3.6 Learning rate

Learning rate is a very important hyperparameter in training deep learning algorithms. It decides how fast an algorithm learns. A smaller learning rate takes more epochs whereas a large learning rate takes a few epochs to reach the optimal value of the function. There is a high chance of fluctuation/divergence from reaching the minimum when the learning rate is higher. Figure 3.7 depicts the importance of the learning rate.

3.3.7 Optimization

Optimization is the iterative process of adjusting hyperparameters to minimize or maximize the objective function by using optimization techniques.

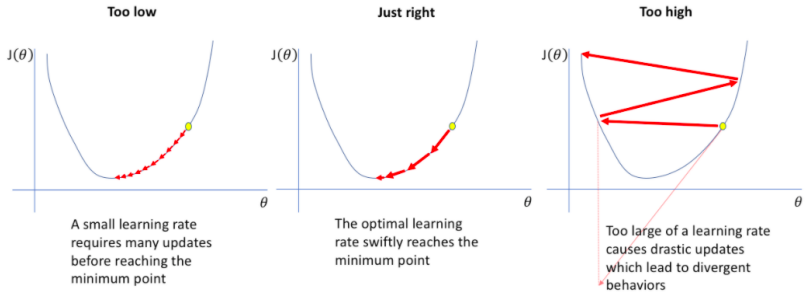


Figure 3.7: Importance of learning rate
(JORDAN, 2018)

Hyperparameters like learning rate, number of epochs, decay factors, batch size are defined before training the model. Model parameters like weights and bias are different from hyperparameters because they are modeled and updated during the training process. The optimization technique aims to train an accurate model with less error rate. The following are the common optimization techniques for most of the machine learning models (Sra et al., 2012).

Gradient Descent

Gradient Descent is the minimization algorithm that minimizes the given function. It finds the local minima of a differentiable function. The limitations of this technique include the inability of the model to find global minima if it gets stuck in local minima and it takes more computational power to train the large dataset at once.

Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent is another optimization technique that solves the problem faced by gradient descent. It does not use the whole dataset at once but instead, it uses the small sample from the large dataset and train the model by using a small dataset. In this way, the time complexity is reduced significantly when compared to gradient descent.

Adam

Unlike the SGD algorithm which maintains a constant learning rate for all the parameters during the training, **Adaptive Moment Estimation**

(Adam) is another optimization technique that adapts the learning rate for each parameter during training. Adam's technique combines the benefits of two optimization algorithms Adaptive Gradient Algorithm (AdaGrad) and Root Mean Square Propagation (RMSProp)(Kingma and Ba, 2014).

3.3.8 Regularization

Overfitting happens when the model tries to memorize the data during training and performs poorly on unseen or test data.

Underfitting occurs when the model is not able to capture the relationship between the input data and target data. This leads to poor performance of the model on training data as well as unseen data.

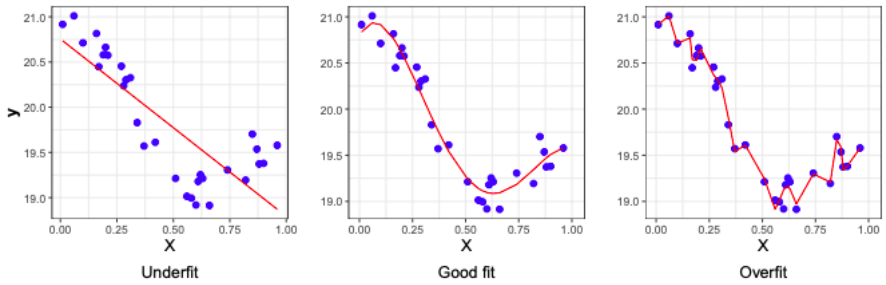


Figure 3.8: Underfit, Good fit, and Overfit
(Kubat, 2017)

Regularization is a technique to avoid model overfitting. Thus it helps to increase the accuracy of the model by providing a good generalization of the given data. Good generalization indicates that the model can make better predictions even on unseen data which is the important behavior of any learning algorithm.

3.4 Deep Reinforcement Learning

Deep Reinforcement Learning is a combination of deep neural networks with a framework of reinforcement learning which helps intelligent machines or software agents to achieve their goal or reward. The 'Deep' in the Deep RL indicates multiple deep neural networks which resemble the structure of the human brain (Henderson et al., 2018b). The framework of Deep RL is depicted in Figure 3.9. The following sections describe the basic terms used in RL.

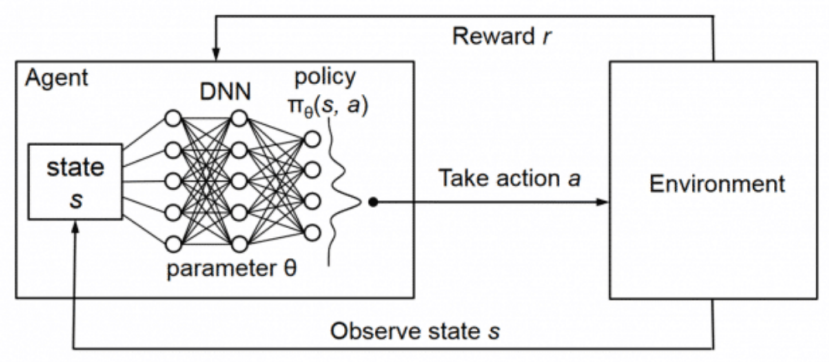


Figure 3.9: Deep Reinforcement Learning framework
(Mao et al., 2016)

(i) **Agent**: Agent is an algorithm that is responsible to take actions based on the rewards or punishments it received before. It learns to take corrective actions by a series of many trial and error steps to achieve a good reward.

(ii) **Environment**: Environment can be a task, game, or simulation with which the agent interacts and tries to solve that task. The agent gives its current state, action as inputs to the environment and receives the reward/feedback, next state in return from the environment.

(iii) **State**: States in RL represents the complete description of the environment. A state s_t is the current situation of the agent in the given environment whereas s_{t+1} is the next state of the agent. The environment takes the current state and gives the next state to the agent to solve the task.

(iv) **Action**: Each RL environment has its own set of **actions**. The set of all possible actions in an environment is called action space. There can be discrete action spaces and continuous action spaces depending on the environment.

(v) **Policy**: Policy is the strategy/algorithm followed by an agent to take actions at a given state which results in the highest reward. This policy part decides whether an RL algorithm is model-based (Section 3.4.1)

or model-free (Section 3.4.2).

$$a_t = \mu\left(s_t\right) \tag{3.3}$$

where:

a_t = Action at time t
μ = Policy followed by agent
s_t = State at time t

(vi) **Reward**: Reward is the feedback given by the environment based on the actions taken by the agent at a given state. Feedback can be either positive or negative. Agent's goal is to take actions that constitute maximum cumulative reward by exploring and exploiting the environment.

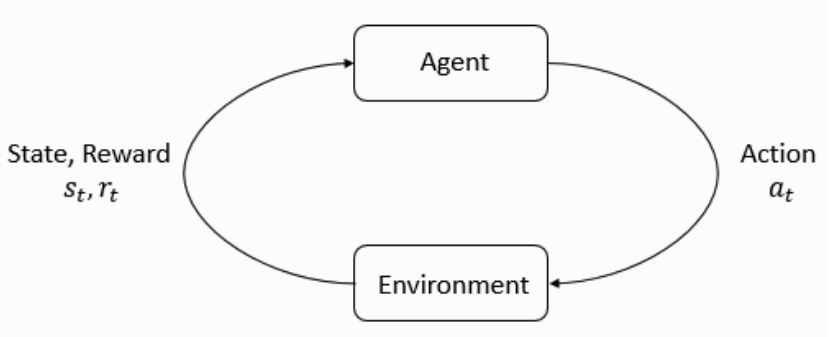


Figure 3.10: Agent-Environment interaction loop
(Sphinx, 2018)

(vii) **Exploration and Exploitation**: In the process of solving an RL task, if the agent tries to visit new states or take new actions then that is called exploration of the sample space. When the agent always takes the known actions which already gave good rewards in the past, then that is said to be exploitation.

(viii) **Discount factor**: It is the weight factor given to the future rewards relative to the immediate rewards that are achieved by the agent. In general, the discount factor is represented as γ.

(ix) **Markov Decision Process (MDP)**: MDP is a framework used to model the reinforcement learning environment. The objective of MDP is to find an optimal policy for the decision-maker (agent) (Polydoros and Nalpantidis, 2017). It is formally represented using a tuple as follows:

$$[S, A, P(s' \mid s, a), R(s, s', a), \gamma] \tag{3.4}$$

where:

S	= Set of states
A	= Set of actions
s	= Current state
a	= Current action
s'	= Next state
$P(s' \mid s, a)$	= Probability of transition to go to next state s' given s, a
$R(s, s', a)$	= Expected reward at state s'
γ	= Discount factor

Markov property (Frydenberg, 1990) states that the future states are dependent only on the current state. The state transition function $P(s' \mid s, a)$ in MDP will satisfy the Markov property.

3.4.1 Model-Based Reinforcement Learning

If a Reinforcement Learning environment makes use of any machine learning model to decide on which action to be taken by the agent, then that is called a model-based RL environment or indirect RL algorithm (Atkeson and Santamaria, 1997). Examples of machine learning models include neural networks, random forest, gradient boost, etc. Thus the machine learning model learns the transition dynamics of the environment resulting in deriving the optimal actions that result in a maximum cumulative reward. Finally, the resultant optimal policy learned by the model is projected into a real-world environment. Figure 3.11 illustrates the pipeline of model-based RL (Polydoros and Nalpantidis, 2017). Two important approaches used to solve model-based RL tasks are **value function** and **policy search**.

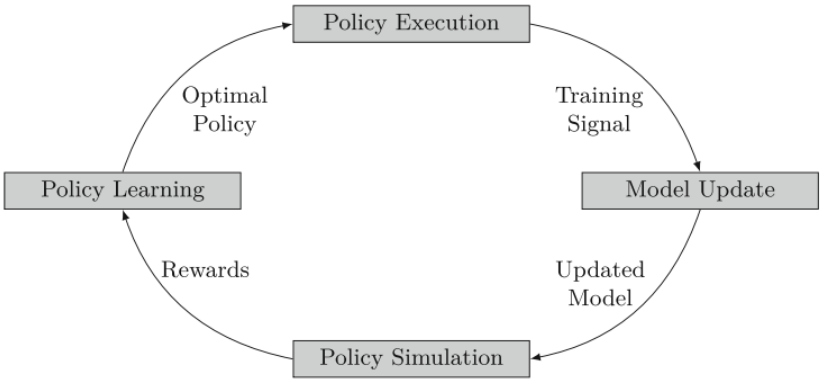


Figure 3.11: Model-Based Reinforcement Learning
(Polydoros and Nalpantidis, 2017)

In **value function approaches**, an optimal state-value or action-value is estimated to derive the optimal actions for each state. The state-value function returns the cumulative reward starting from the state s by following a policy π whereas the action-value function returns the cumulative reward starting from the state s, by following a policy π and taking action a. Then the policy which maximizes the cumulative reward is derived from the obtained optimal value function. The optimal policy is constructed indirectly by finding the actions that maximize the value function at each state (Polydoros and Nalpantidis, 2017). The value function methods are categorized into four different classes as in Figure 3.12.

- **Dynamic Programming** (DP) methods
- **Differential Dynamic Programming** (DDP) methods
- **Temporal Difference Learning** (TDL) methods
- **Monte Carlo** (MC) methods

Value functions depend on the immediate reward of the future states $R(s, s', a)$ and discounted value functions $\gamma V^{\pi}(s')$, $\gamma Q^{\pi}(s', a')$ weighted by the transition probabilities $P(s' \mid s, a)$. The optimal policy π^* is obtained when the value function is maximized. Thus the Bellman's equation for optimal state-value function V^* and optimal action-value function Q^* are represented in a recursive form as follows:

$$V^{\pi_*}(s) = \max_{a \in A(s)} \sum_{s'} P(s' \mid s, a) \left[R(s, s', a) + \gamma V^{\pi_*}(s') \right] \tag{3.5}$$

$$Q^{\pi*}(s,a) = \sum_{s'} P\left(s' \mid s,a\right)\left[R\left(s,s',a\right) + \gamma \max_{a'} Q^{\pi*}\left(s',a'\right)\right] \tag{3.6}$$

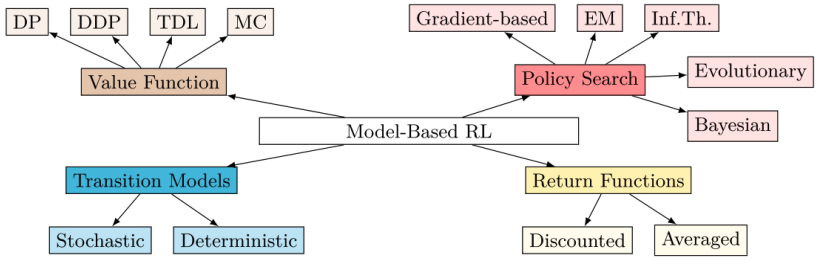


Figure 3.12: Different approaches for solving model-based RL tasks (Polydoros and Nalpantidis, 2017)

Among the above four classes, dynamic programming methods are very popular in model-based RL. They provide two important iterative algorithms to find the optimal policy which are known as **policy iteration** and **value iteration**.

1. Policy iteration: The first step of this algorithm is policy evaluation whereas the second step is policy improvement. In the policy evaluation step, the state or action-value function is computed in an iterative process for each state until the convergence is obtained. In the policy improvement step, the best action is selected for each state. Initially, the algorithm starts with a random policy, then the policy evaluation step is followed by the policy improvement step (Depraetere et al., 2014). Updation of policy takes place only after the evaluation step is converged.

2. Value iteration: Unlike the policy iteration algorithm which has two steps, the value iteration algorithm has a single step to find the optimal values. Firstly, it starts with a random value function then iteratively updates the state value function by taking the maximum value of all possible actions in each state. Thus the value function is updated in a single step at each iteration. Value iteration algorithm requires more iterations to converge whereas the policy iteration requires fewer iterations to converge. (Bakker et al., 2006)

In **policy search approaches**, the optimal policy is derived directly by updating the parameters of the model. Policy search methods use different algorithms to find the optimal parameters that constitute the optimal policy

as follows:

- **Gradient-Based**
- **Expectation-Maximization** (EM)
- **Information Theory**
- **Evolutionary Computation**
- **Bayesian Optimization**

The baseline paper and this thesis work utilize the evolutionary computing strategy for deriving the optimal policy by finding the optimized weights of an artificial neural network. The working principle of evolution strategy is clearly explained in Section 3.8.

Transition models indicate the mapping from the current state s to the next state s' by taking action a at a given time step. These models can either be stochastic or deterministic in nature. The stochastic transition model utilizes probability distributions for predicting the future states so the predictions of the model will always be different for a given state and action. The deterministic transition model does not use any random variable for predicting the future states thus the predictions of the model will always be the same for a given state and action. Some of the approaches used for learning stochastic transition models include Gaussian processes, deep neural networks whereas physics-based, linear regression, decision trees methods are used for learning deterministic transition models.

$$f(s, a) \mapsto s' \tag{3.7}$$

The aggregation of rewards or punishments obtained by an agent over a whole episode is computed using **return functions**. These functions can either be discounted (Equation (3.8)) or averaged (Equation (3.9)). Discounted return functions apply more weight on the early rewards and exponentially decrease weight on the future rewards whereas the averaged return functions give the average of all the obtained rewards during an episode.

$$R = \sum_{t=0}^{\infty} \gamma^t r\left(s_t, a_t\right) \tag{3.8}$$

$$R = \frac{1}{T} \sum_{k=0}^{T-1} r\left(s_t, a_t\right) \tag{3.9}$$

Both the selection of transition models and return functions play a major role in the performance of model-based RL.

3.4.2 Model-Free Reinforcement Learning

If a Reinforcement Learning environment does not use any machine learning model but rather use a trial-and-error approach to decide on which action to be taken by the agent, then that is called model-free RL environment or direct RL algorithm (Atkeson and Santamaria, 1997). These algorithms do not require any prior knowledge of transition models and can be easily implemented. One of the disadvantages of these algorithms is the slow convergence rate. Temporal Difference Learning (TDL) and Monte Carlo (MC) methods do not require any model, so they are widely used in model-free RL.

3.5 Deep Generative Models

Deep Generative Models focus on modeling the probabilistic distribution of the given input data. By learning the probability distribution of the dataset, it is possible to generate new data from the obtained distribution. In general, generative models are applied to unsupervised learning tasks. The probability of unlabelled data x is observed as $P(x)$ and in the case of labeled dataset generative model observes $P(x|y)$, the probability of observation x given its label y. When the deep neural networks are combined with the generative modeling architectures they become deep generative models. The most fundamental architectures of deep generative modeling are discussed in the following sections.

3.5.1 Autoencoder

Autoencoder is an unsupervised learning model where the goal of training is to better approximate the input data. The applications include dimensionality reduction, image denoising, etc.,(Wei et al., 2020). It has an encoder-decoder architecture where the encoder encodes the original data and the decoder decodes the encoded data. The encoder accepts the high dimensional data X as input and compresses it into low dimensional latent representation Z. Decoder takes the latent representation Z as input and reconstructs or decompresses it into original domain X'. The performance of

the autoencoder is evaluated on how well the latent representation captures the important features of raw input data (Kingma and Welling, 2013).

$$X' = f(Z) = f(h(X)) \tag{3.10}$$

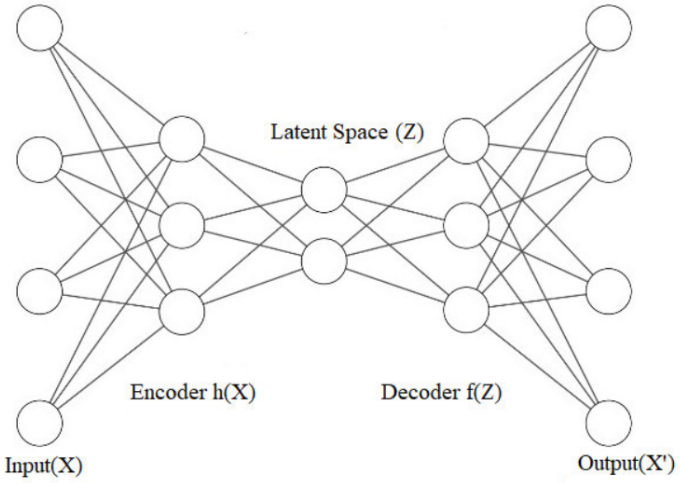


Figure 3.13: Architecture of Autoencoder
(Wei et al., 2020)

3.5.2 Variational Autoencoder (VAE)

Variational Autoencoder is the enhanced version of autoencoder which adds the Bayesian component to learn the parameters ($\mu(x)$ and $\Sigma(x)$) of the distribution of data (x). Autoencoders are able to accomplish dimensionality reduction but they cant generate new data. This limitation can be dealt by VAE successfully. The section (Section 5.2.1) discusses the VAE in more detail. Figure 3.14 depicts the architecture of VAE.

3.6 Deep Neural Networks (DNN

The architecture of **deep neural networks** is inspired by the functioning of the human brain which uses multiple neurons to process the information for solving a particular task (Sejnowski, 2020). Deep neural networks process multiple layers of data between input and output layers to solve a task. Each layer constitutes numerous nodes like neurons. The more complex task needs more layers to process which makes the network 'deep'. DNN's are playing a major role in real-world applications like speech recognition, multi-object

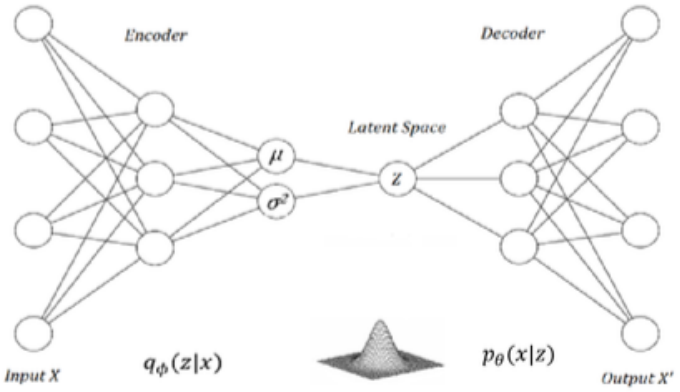


Figure 3.14: Architecture of Variational Autoencoder
(Wei et al., 2020)

detection, fraud detection, creative thinking, making predictions, unmanned aerial vehicles, self-driving cars, etc. The following sections discuss the different types of deep neural networks which are useful to solve a specific task.

3.6.1 Artificial Neural Networks (ANN)

ANNs are also known as feed-forward neural networks because the input information is only processed in the forward direction. It has 3 layers: input layer, hidden layer, and output layer. Each layer consists of multiple perceptrons. The input layer takes the inputs, hidden layers processes the inputs, and the output layer generates the results. These are also called **Universal Function Approximators** as they are capable to learn any nonlinear function (Bre et al., 2018). This approximation is possible because of the activation function which is responsible to introduce non-linear properties to the network.

The limitations of ANN include the increase of trainable parameters when the image size is large while trying to solve the image classification task. Also, it is not possible to capture the sequential information of input data using ANN. The following section introduces the specific architectures which are useful to overcome the above limitations of ANNs.

3.6.2 Convolutional Neural Networks (CNN)

The architecture of a **Convolutional Neural Network** is specially designed to deal with the tasks related to image and video processing. CNN's play

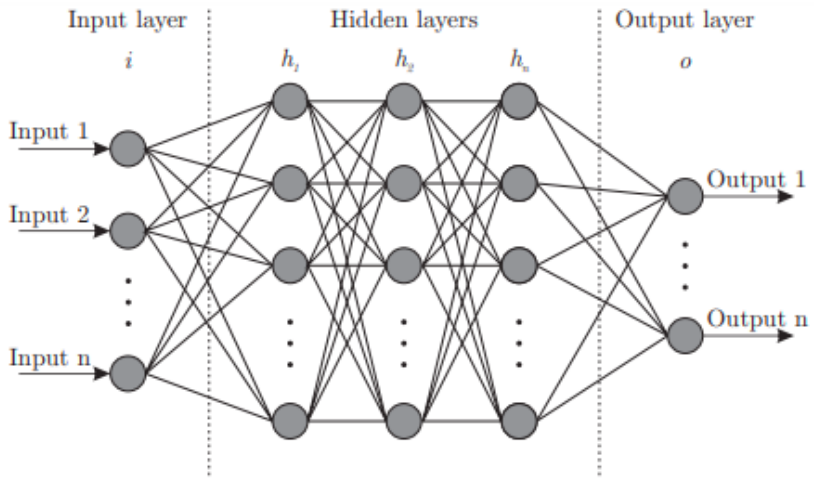


Figure 3.15: Architecture of ANN
(Bre et al., 2018)

a major role in computer vision applications. The CNN architecture can effectively learn the spatial and temporal dependencies of input image data. Shared weights are used across the layers of the network which drastically reduces the number of trainable parameters (Yamashita et al., 2018). The main building blocks of the convolutional neural network include convolution layer, pooling layer, and fully connected layer which are discussed in detail in the following sections. The architecture of CNN has depicted in Figure 3.18

3.6.2.1 Convolution Layer

Convolution layer performs convolution operation on the given input image to automatically extract the meaningful features. The main element of this convolution operation is called 'filter' or 'kernel' which is responsible to extract the features and generating a feature map. In general, the size of the kernel is small when compared to the size of the input image. The machine understands the input image as the array of numbers i.e each pixel of an image represents a number between 0 and 255. The kernel which is again a small array of numbers slides over the whole input image from the top left corner to the bottom right corner. An element-wise product is calculated between each element of the kernel and the input element then they are summed up to generate one element of the feature map. This process is repeated using multiple kernels to generate multiple feature maps. Multiple

kernels are used in order to extract different characteristics (edge detection, sharpen or blur the image) of input data.

The outputs from this linear convolution operation are then passed through the non-linear activation function to capture the non-linear behavior in the input data.

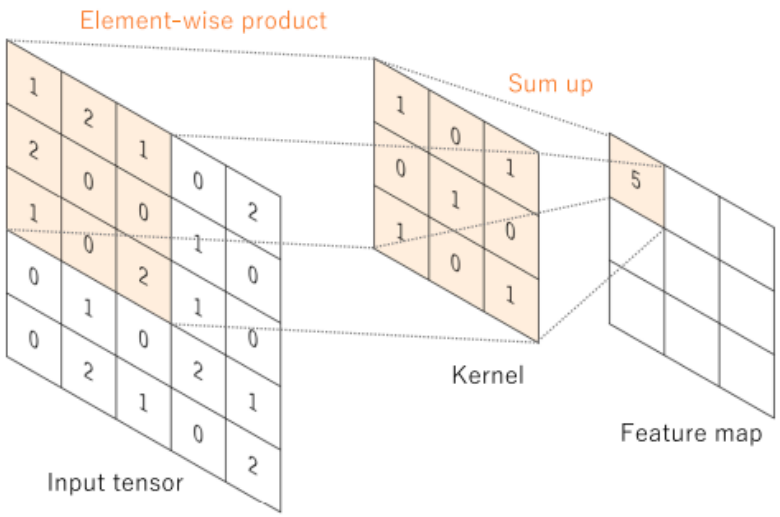


Figure 3.16: Convolution operation
(Yamashita et al., 2018)

Hyperparameters

The following are some of the important hyperparameters used in CNNs:

Size of Kernel

The size of the matrix with weights which is used to convolve the input image is the size of the kernel. The size of the kernel is chosen based on the number of different features to be extracted.

Stride

When the kernel is sliding over the input image, the distance between one kernel position and the next kernel position is called stride. The size of the stride decides the size of the feature map. Larger strides downsample the

feature maps.

Padding

Padding is a technique of adding zeros column-wise and row-wise at each side of an input image. This is helpful to keep the same in-plane dimensions of feature maps. It is also helpful to persist the edge features of the input.

3.6.2.2 Pooling Layer

Pooling layer is used in downsampling the dimensionality of generated feature maps from the convolution layer. Max-pooling and Mean-pooling are the two famous operations in the pooling layer. Max-pooling is performed by taking the maximum value from the patch of feature map whereas the Mean-pooling is performed by taking the average value as shown in Figure 3.17

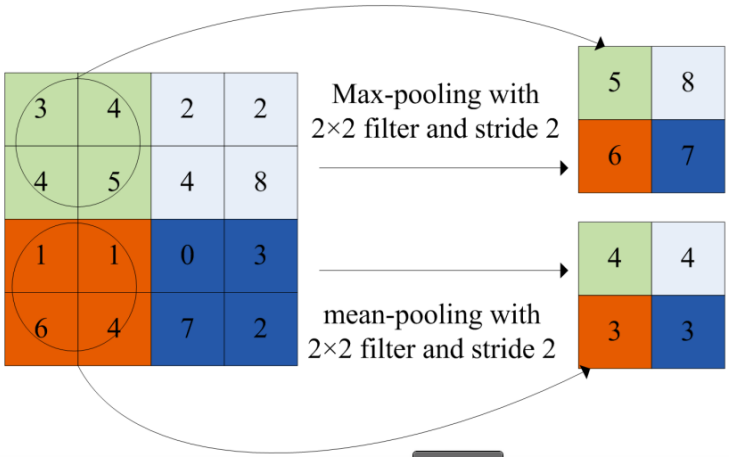


Figure 3.17: Max-pooling and Mean-pooling (Mao et al., 2016)

3.6.2.3 Fully Connected Layer

The output from the multiple convolution layers and pooling layers is finally given to the fully connected layer where it flattens the feature maps to one-dimensional vectors. This fully connected layer is also called a dense layer because each input is connected to each output using a learnable weight. The

final layer of the fully connected layer gives the probabilities of each target class for the image classification task.

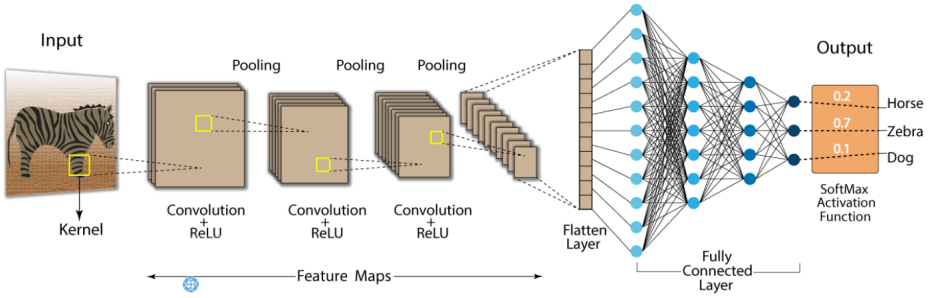


Figure 3.18: Convolutional Neural Network (Swapna, 2021)

3.6.3 Recurrent Neural Networks (RNN)

Recurrent Neural Networks provide a unique architecture to effectively model the sequential data (Schuster and Paliwal, 1997). Examples of sequential data include audio, video, text, etc. Speech recognition, natural language processing, machine translation are some of the famous applications where recurrent neural networks excel.

In general, the hidden unit in ANN takes the input and gives the output. But the hidden unit h^t in RNN architecture accepts the current input x^t along with the information from previous hidden states h^{t-1}. The output vector $\hat{y}^t$ builds a predictive distribution by taking the current input along with hidden states with a collective input of $[x^1, x^2, x^3, ...x^t]$ in order to predict the next input x^{t+1} for the network (Graves, 2013). A looping mechanism is added to the normal ANN to make it RNN. This looping mechanism allows the flow of information from the previous time step to the current time step. The way the input data is fed into the network is also different in RNN. ANN accepts the complete input at once whereas the RNN is fed with one input at a time. The following Figure 3.19 depicts the unfolded architecture of RNN. The output from the current time step predicts the input for the next time step as shown in the figure.

where:

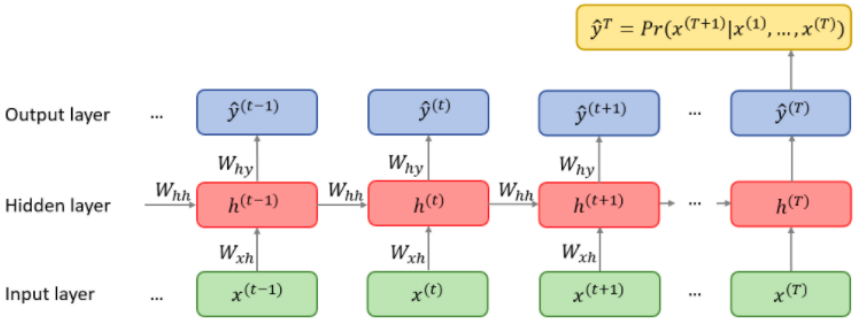


Figure 3.19: Unfolded computational graph of RNN (Plesiak, 2020)

$x^{(t)}$,$h^{(t)}$,$y^{(t)}$ = Input, Hidden, Output states at time t
W_{xh} = Weight matrix connecting inputs to hidden state
W_{hh} = Weight matrix connecting recursively to hidden states
W_{hy} = Weight matrix connecting the hidden state to output

These weight matrices or model parameters are shared across the whole network and updated using a special algorithm called **Backpropagation Through Time (BPTT)**. This algorithm recursively computes the gradients at each time step to obtain the single loss at time t. Then these single losses are summed up to get the total loss. While updating the weights of the network RNN architecture faces a vanishing and exploding gradient problem.

Vanishing gradients occurs when the multiplication of small derivatives across the hidden layers leads to shrinking the gradient near to zero.

Exploding gradients occur when the multiplication of large derivatives across the hidden layers leads to exploding the gradient to a very large value.

To address the vanishing gradient problem, gated RNNs are introduced in the literature (Chung et al., 2015). The following sections discuss the working of gated RNNs.

3.6.3.1 Long Short Term Memory (LSTM)

The small gradient updates in the RNN layers result in no learning thus forgetting the information from the early layers. This is referred to as short-term memory. To avoid this problem the architecture of LSTM (Hochreiter and Schmidhuber, 1997) contains internal gates namely forget gate, input gate,

and output gate which controls the flow of information into the network. These gates allow only the important information to flow into the network while discarding the unnecessary information. Thus the network can remember the long sequences. The functionality of each gate is as follows:

- Forget gate decides what information to keep from the previous steps.
- Input gate decides what information to add from the current step.
- Output gate decides what information to be passed to next the hidden state.

The architecture of LSTM is further discussed in detail in Chapter 3.

3.6.3.2 Gated Recurrent Units (GRU)

GRU is another variant of RNN which also tries to address the vanishing gradient problem. It consists of two gates called reset gate and update gate.

- Update gate decides what information to discard and what information to keep. This is similar to forget and input gate in LSTM.
- Reset gate decides how much information to forget from the past.

3.7 Mixture Density Networks (MDN

In general, ANNs predict a single output for the target class assuming that the target data distribution is unimodal i.e one continuous value for regression task and one discrete value for classification task. What if the underlying target distribution is multimodel?. Then the predicted single target value is not sufficient to well describe the target data. This problem introduces a unique architecture called **Mixture Density Network** (Bishop, 1994) which parametrizes the mixture of probability distributions of the target data instead of predicting one single output value. MDN's are being used in various applications like speech recognition by Apples Siri (SiriTeam, 2017), to generate artificial handwriting (Graves, 2013).

Formally, MDN represents the conditional probability $p(y \mid x)$ of target data as

$$p(y \mid x) = \sum_{i=1}^{m} \alpha_i(x)\phi_i(y \mid x) \tag{3.11}$$

where:

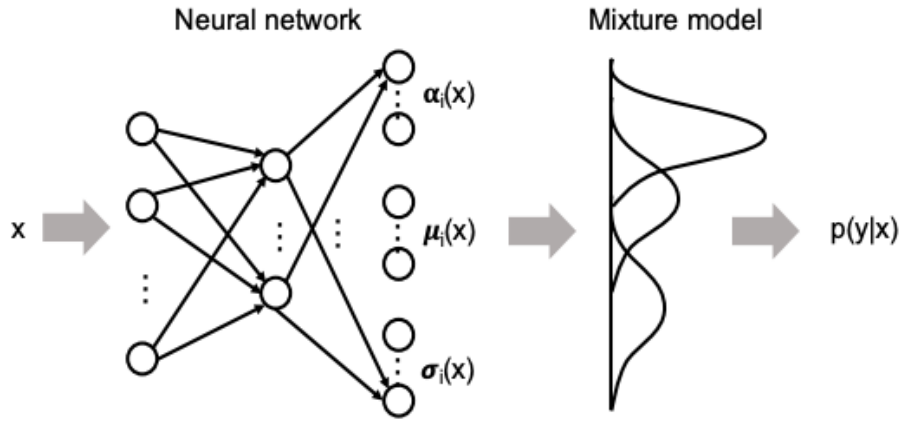


Figure 3.20: Mixture Density Networks
(Vossen et al., 2018)

i = Index of mixture component
m = Total number of mixture components (distributions) per output
$\alpha_i(x)$ = Mixing coefficients conditioned on input x
$\phi_i(y \mid x)$ = Kernel function

The Kernel function is nothing but a corresponding distribution that is selected depending on the task or application. Gaussian kernel function is defined as

$$\phi_i(y \mid x) = \frac{1}{\sqrt{2\sigma_i(x)^2\pi}} \exp\left(-\frac{(y-\mu_i(x))^2}{2\sigma_i(x)^2}\right) \tag{3.12}$$

where:

$\mu_i(x)$ = Mean conditioned on input x
$\sigma_i(x)$ = Standard deviation conditioned on input x

Parameters are updated using the backpropagation technique and the error is computed by taking the negative logarithm of the likelihood as follows:

$$E(y, \hat{y}) = -\ln \mathcal{L}(\hat{y} \mid x) = -\ln p(\hat{y} \mid x) \tag{3.13}$$

3.8 Evolution Strategies (ES) for RL

In a traditional RL setting, the agent aims to learn a policy that tends to give the highest cumulative reward. The policy is a neural network that takes

the current state of the game as an input and computes the probability of taking any of the allowed actions in i.e action space. The Backpropagation technique is used to update the parameters of a policy to take a precise action in a given state to get a high reward.

Evolution Strategies are black-box optimization algorithms used to find optimal solutions in continuous search space problems (Majid et al., 2021). They are also known as gradient-free optimization techniques because the objective function need not be differentiable and continuous(Salimans et al., 2017). ES algorithms work by first sampling a population of individuals, then selecting the best-performed individuals to further sample the future generations. The working of this algorithm is better described using the below Figure 3.21.

- Initialization: Initially, the ES algorithm generates a population P which consists of individuals.
- Parent selection: A small set of individuals are selected from the population to function as parents during the recombination step.
- Recombination: Two or more parents combine to generate a mean for the new/offspring generation.
- Mutation: A small amount of noise is added to the recombination results.
- Evaluation: A fitness score is assigned to each candidate solution.
- Survivor Selection: The best μ individuals are selected based on the evaluated score to form the population for the next generation. Then the algorithm repeats all the above steps until an optimal solution is found.

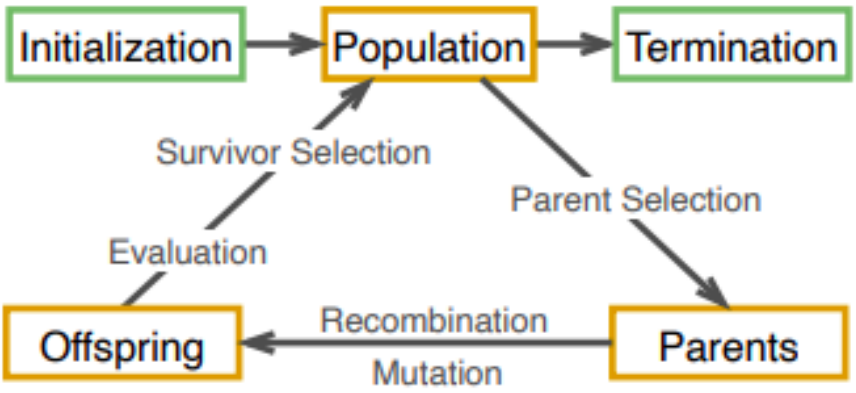


Figure 3.21: Evolution Strategy
(Majid et al., 2021)

Chapter 4

Methods and Metrics

This chapter provides a detailed study of baseline and proposed methodologies. Section 4.1 discusses all the components of the baseline architecture (Ha and Schmidhuber, 2018) in detail. The name given by the authors for baseline architecture is 'world model'. Section 4.2 illustrates the improved architecture and will forth on be denoted as 'advanced world model' because the improved model tries to increase the performance of the baseline model by using advanced deep learning algorithms.

4.1 Baseline Architecture

The following subsections explain the different components of the world model which served as a baseline for the improved architecture. The results from the proposed pipeline are then compared with the baseline methodologies in order to evaluate the performance of the proposed methodology.

4.1.1 World Model Architecture

The architecture of world models constitutes of three components such as vision (V), memory (M), and controller (C). Each component is responsible for each specific task. The task of the vision (V) model is to compress the high dimensional input to a low dimensional latent vector. The memory (M) model is responsible to take all the compressed previous frames from V as input and predict the next frame as output. Finally, controller (C) also called the decision-maker takes the low dimensional latent vector from V along with the predicted next frame from M then decides what action to be taken in order to maximize the expected final reward. The following subsections discuss each of these components in detail. The architecture of world models is

illustrated in the following Figure 4.1

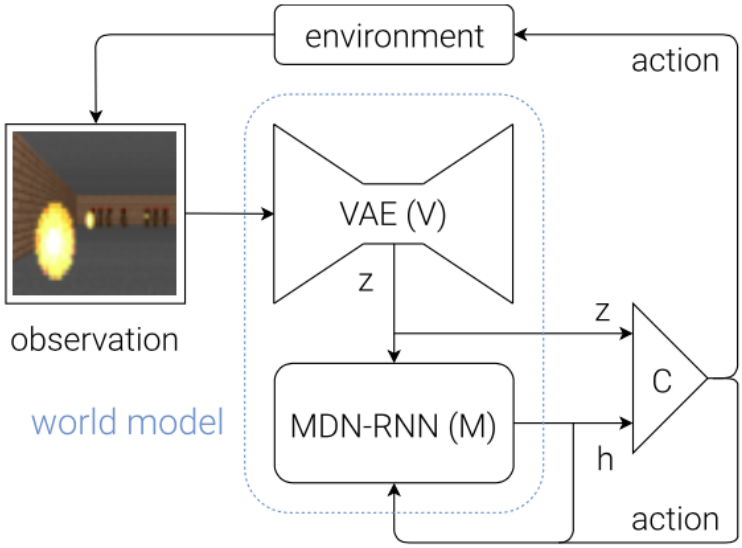


Figure 4.1: Flow diagram of World Models
(Ha and Schmidhuber, 2018)

4.1.1.1 Vision (V) Model

Variational AutoEncoder (VAE) (Kingma and Welling, 2013; Rezende et al., 2014) is chosen as a vision (V) model in the baseline architecture. The environment which is used to train the complete world model consists of multiple high dimensional input observations. These observations are two-dimensional image frames that are collected from a video sequence. Vision (V) model takes the high dimensional image frame at each time step as input and gives the compressed representation of each observed frame as output.

Variational AutoEncoder (VAE)
This part discusses the complete architecture of VAE. In general, VAE was inspired by Bayesian inference (Kingma and Welling, 2013; Doersch, 2021). The idea behind the VAE architecture is to model the underlying probability distribution of input data X to sample new data from the modeled distribution. VAE is trained in such a way that the produced new data should be realistic and similar to training data.

Latent spaces generated by VAE are continuous unlike the latent spaces of autoencoder which are deterministic (Pu et al., 2016). Enforcing a prior on latent space by VAE is responsible for this continuous feature of latent space of VAE which enables easy random sampling and interpolation. Autoencoder maps the input onto a single point in space whereas the encoder of VAE maps the input onto a normal distribution. The latent vector Z in VAE is sampled from two latent variables Z_μ and Z_σ as in Figure 3.14. Then the sampled vector Z is given to the decoder in order to predict the reconstructed input $\hat{X}$. The latent vector Z is made to follow standard normal distribution by training the parameters Z_μ, Z_σ in such a way that $Z_\mu \to 0$ and $Z_\sigma \to 1$.

There is one issue when the latent vector Z is sampled from two latent variables Z_μ and Z_σ. As this sampling is stochastic in nature gradients cannot be backpropagated, so that latent variables Z_μ and Z_σ cannot be learned. In general, the backpropagation algorithm wants the nodes to be deterministic to pass the gradients iteratively to update the parameters by using the chain rule. Authors (Kingma and Welling, 2013) introduced an important trick to overcome this issue which is called the **reparameterization trick**. By using this trick the stochastic node Z becomes the deterministic node thus allowing the parameters to be learned. Figure 4.2 illustrates the architecture of VAE. Reparameterization trick is implemented using the Equation (4.1) as follows:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \text{ where } \boldsymbol{\epsilon} \sim \mathcal{N}(0, 1) \tag{4.1}$$

where:

μ = fixed mean vector
σ = fixed standard deviation vector
ϵ = auxiliary independent random variable drawn from unit Gaussian
$\odot$ = element-wise multiplication

VAE aims to optimize two loss functions. One is reconstruction loss which ensures that the generated images by the decoder are similar to the images in the training dataset. The second loss term is the Kullback-Leibler (KL) Divergence loss which measures the divergence between a pair of probability distributions and ensures that the sampled latent vectors are close to the standard normal distribution. The value of the divergence term is minimized if the distribution of generated sample $q_\phi\left(z \mid x^{(i)}\right)$ matches the prior distribution $p_\theta(z)$. The quantity which is optimized in VAE is called Evidence Lower Bound (ELBO) or variational lower bound. Equation (4.2) for the ELBO is

given as follows where the first term in the RHS represents reconstruction error and the second term represents KL divergence error.

$$\mathcal{L}(\theta, \phi) = E_{q_\phi(z|x)} \left[\log p_\theta(x \mid z)\right] - D_{KL} \left(q_\phi(z \mid x) \| p_\theta(z)\right), \tag{4.2}$$

where:

θ : generative parameters
ϕ : variational parameters
x : sample data point
z : latent representation
$p_{\boldsymbol{\theta}}(\mathbf{z})$: prior
$p_{\boldsymbol{\theta}}(\mathbf{x} \mid \mathbf{z})$: probabilistic decoder
$q_{\boldsymbol{\phi}}(\mathbf{z} \mid \mathbf{x})$: probabilistic encoder

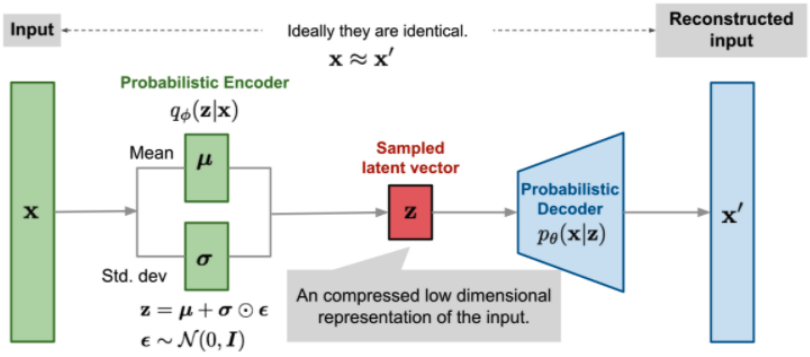


Figure 4.2: Illustration of VAE
(Weng, 2018)

4.1.1.2 Memory (M) Model

Recurrent Neural Network combined with a Mixture Density Network (MDN-RNN) is used as a memory (M) model which acts as a predictive model of the future Z_{t+1} vectors by taking the previous latent vectors $Z_1, ...Z_t$ as input from the vision (V) model. Ha and Schmidhuber (2018) trained the RNN to output the probability distribution $P(Z_{t+1})$ of the next latent vector instead of the deterministic prediction of Z_{t+1} because of the stochastic nature of different complex environments. Figure 4.3 depicts the memory model where it takes the action (a_t) at time t, latent vector (z_t) at time t, and hidden vector (h_t) at time t as inputs and output the probability distribution as $P(z_{t+1})$.

The temperature parameter (τ) is used to control the uncertainty of a model during sampling.

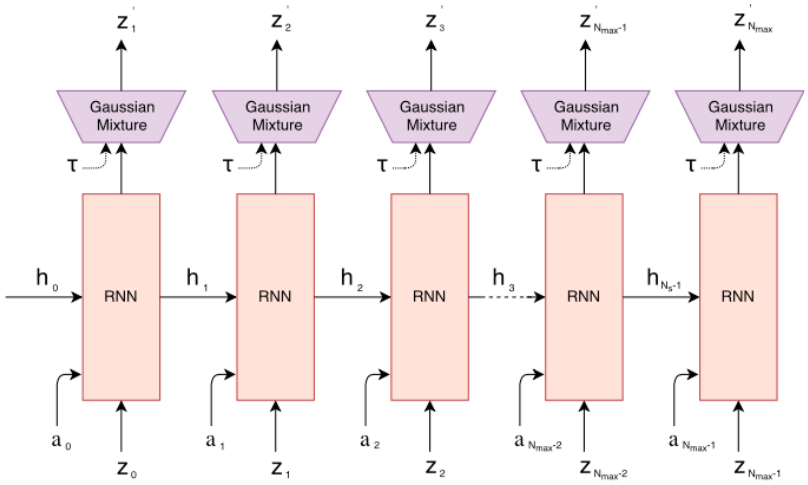


Figure 4.3: Memory (MDN-RNN) model
(Ha and Schmidhuber, 2018)

Long Short Term Memory (LSTM)

LSTM architecture (Hochreiter and Schmidhuber, 1997) is used in memory (M) model. It is one of the famous architecture of RNN introduced to solve the vanishing gradient problem faced by RNN. This section explains the LSTM in detail. The main idea of LSTM is to use a memory cell that is responsible to model the long-term dependencies effectively. With the help of different gates in the architecture, the memory cell decides which information to keep from the current state and which information to discard.

The three important gates in the LSTM architecture are the forget gate, the input gate, and the output gate. First, the forget gate (f_t) takes the input from the current time step (x_t) and latent information from the previous time step (h_{t-1}) then decides which information to discard from the cell state (C_{t-1}). This decision is made by using the sigmoid activation function on each value in the cell state. '1' indicates to completely keep the information whereas '0' indicates to completely discard the information. The previous cell state is multiplied with forget gate to forget the unnecessary information.

Next, the input gate (i_t) decides what new information should be stored

in the cell state. This is done by first applying the sigmoid function on the input data to decide what values to update and also the tanh function to get a vector of candidate values ($\tilde{C}_t$) which could be added to the state. Then the point-wise multiplication of i_t and $\tilde{C}_t$ tells how much information is to be updated in the cell state. Then this updated candidate vector $i_t * \tilde{C}_t$ is added to the previous cell state C_{t-1} to make it a new cell state C_t.

Finally, the output gate o_t decides what information should be given as output. The sigmoid function is applied first to decide which parts of the cell are passed to the output and then the tanh activation function is applied to the updated cell state C_t to filter what values to pass to the output. The outputs from the sigmoid function and the tanh function are multiplied to give the final output h_t. In the below equations W, b represents the weight matrix and bias vector for the corresponding layer. Figure 4.4 depicts the architecture of an LSTM cell with internal gates.

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] + b_f\right) \tag{4.3}$$

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right) \tag{4.4}$$

$$\tilde{C}_t = \tanh\left(W_C \cdot [h_{t-1}, x_t] + b_C\right) \tag{4.5}$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \tag{4.6}$$

$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right) \tag{4.7}$$

$$h_t = o_t * \tanh\left(C_t\right) \tag{4.8}$$

Teacher Forcing

In general, sequence prediction models like RNNs use the predicted output from the previous time step y_{t-1} as the input for the current time step x_t during training. In sequence prediction tasks, it is evident that the output predicted at a particular time step is dependent on the current input. So if the predicted output at time step t is erroneous then the whole sequence which will be predicted in future time steps will be wrong. This training strategy leads to slow convergence and instability of the model. To overcome these limitations of standard training in RNNs, the teacher forcing technique is introduced in the literature. (Doya, 1993).

Teacher forcing (Doya, 1993; Williams and Zipser, 1989) is a technique to train the recurrent neural networks by using the ground truth as the input for the network instead of feeding the predicted prior output as input in the next time step. This teacher forcing method has only one difference when

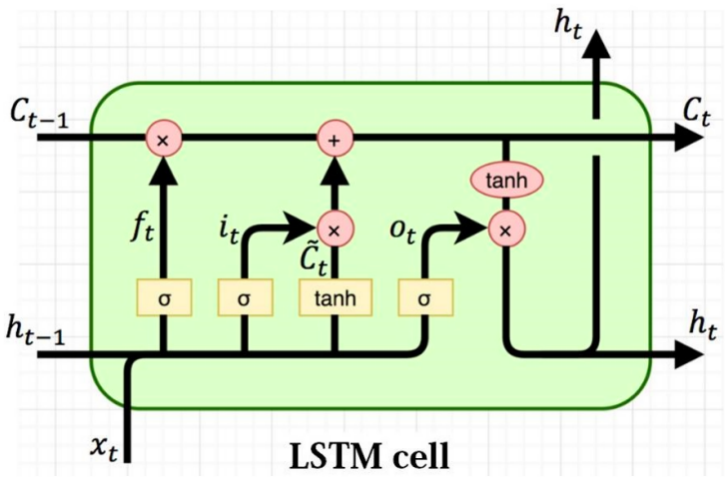


Figure 4.4: LSTM cell with internal gates
(Varsamopoulos et al., 2018)

compared to general training. In both cases, the loss is computed at every time step between the predicted and ground truth, then the cumulative loss is given at the end. After computing the loss, the predicted output will be given as input for the next time step in general training whereas in teacher forcing training, ground truth will be given as input for the next time step by discarding the predicted output. Thus teacher forcing strategy makes the training of recurrent neural networks quick and efficient. Figure 4.5 depicts the teacher forcing strategy. It is like a teacher helping a student while preparing for an exam and during the real exam, student alone has to use his/her learned skills effectively to perform better in the exam. Likewise, the teacher forcing strategy is also used only during training. During the inference or prediction, only the predicted output at the previous time step is given as input for the current time step.

4.1.1.3 Controller (C) Model

Controller (C) model is a simple linear model which takes the compressed latent vector z_t from vision (V) model and predicted latent vector h_t from memory (M) model at time t then outputs the corresponding action a_t at time t to be taken by the agent in order to maximize the cumulative reward.

$$a_t = W_c \left[z_t h_t \right] + b_c \tag{4.9}$$

where:

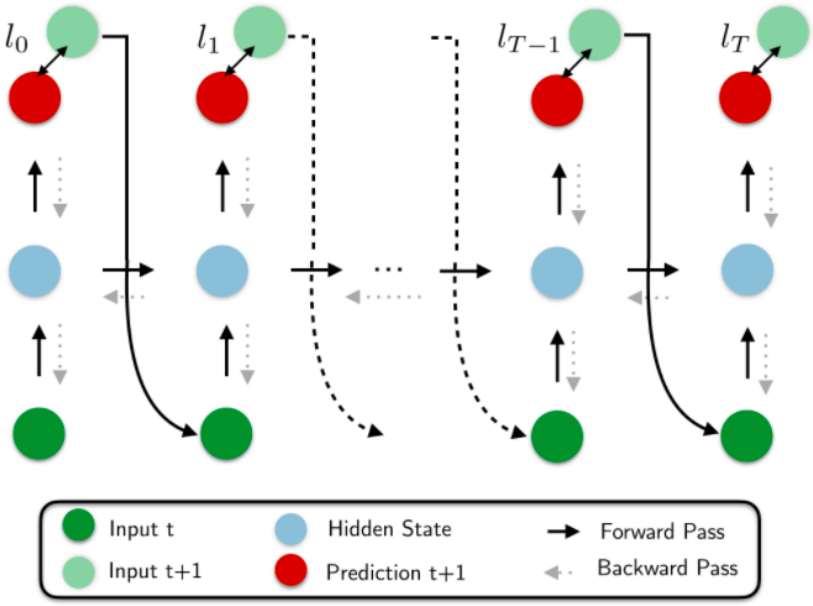


Figure 4.5: Teacher forcing in RNN
(Lange, 2020)

$[z_t h_t]$ = concatenated input vector
a_t = action output vector
W_c = weight matrix
b_c = bias vector

Covariance-Matrix Adaptation Evolution Strategy (CMA-ES)

CMA-ES (Hansen, 2006; Hansen and Ostermeier, 2001) is one of the well known black-box optimization techniques. It is used to optimize the parameters of controller (C) model. Though the gradient based optimization techniques yield best results, literature has proved that CMA-ES work well up to a few thousand parameters (Ha and Schmidhuber, 2018). In general CMA-ES works as follows and illustrated in the Figure 4.6

- In the first step, based on the multivariate normal distribution a random population is generated.
- Best candidates which give the highest cumulative reward are selected from the population. They update the model parameters like mean or covariance matrix of the search distribution.
- Then the new population is generated from the updated distribution.

- All the above steps are repeated untill the convergence is attained.

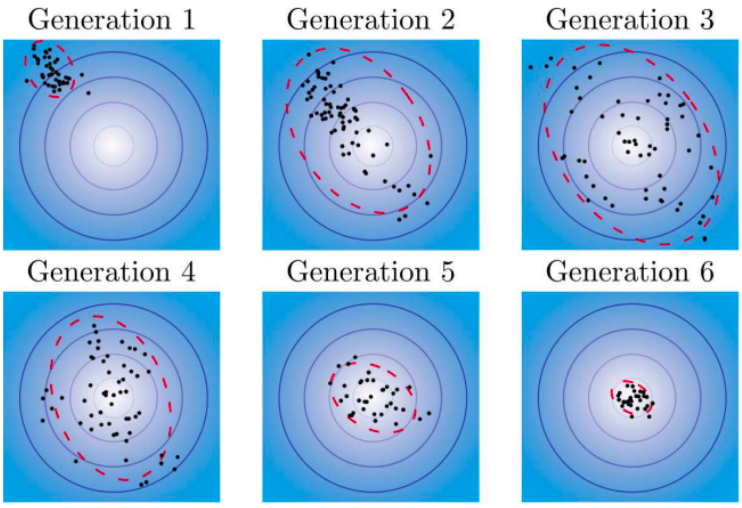


Figure 4.6: Illustration of CMA-ES algorithm
(Tan et al., 2019)

4.2 Improved Architecture

This section explains how the baseline architecture has been improved by using the advanced deep generative models.

4.2.1 Advanced World Model Architecture

Variational Autoencoder (VAE) was used as a vision (V) model in the baseline architecture. Despite of many advantages of VAE like learning smooth latent representations, generating new meaning samples in unsupervised manner etc.,it also has few limitations like encoding some parts of the observation which are not relevant to the given task (Ha and Schmidhuber, 2018), blurry image generation, uninformative latent space, prior is unimodal thus doesn't support mixed data distributions etc,. Tremendous research is going on to mitigate these limitations and literature presents various variations of VAE depending on task requirements with the aim of improving the quality of generated data (Wei et al., 2020). In this thesis work, the aim is to improve the vision (V) model by replacing VAE with advanced deep generative models i.e variations of VAE like InfoVAE (Zhao et al., 2017) and β-VAE (Higgins

et al., 2016; Mathieu et al., 2019). Figure 4.7 depicts the flow diagram of world model with improved vision (V) model making it a advanced world model.

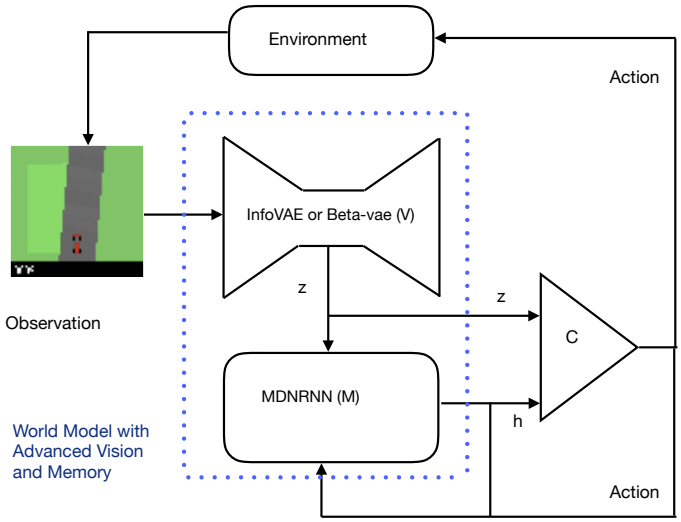


Figure 4.7: Flow diagram of Advanced World Models

4.2.1.1 Advanced Vision Model

Feature learning or feature engineering or representation learning refers to transforming raw input data to useful training data representations. Representation learning learns the useful disentangled features automatically. Disentanglement is an unsupervised learning technique which encodes the input into separate dimensions and enables the representations to break down into narrowly defined variables. A latent vector is called factorial disentangled when the dimensions are independent of each other. To make better usage of the latent information generated from vision (V) model, this research uses MMD-VAE, β-VAE as advanced vision model in the improved architecture. MMD-VAE, β-VAE models are chosen because they are proved to learn the disentangled latent information that can overcome the limitation of baseline vision (V) model i.e uninformative latent information.

Maximum Mean Discrepancy VAE (MMD-VAE):

MMD-VAE (Zhao et al., 2017) has been chosen as a vision (V) model in the advanced world model. It is a member of the **Information Maxim-**

izing Variational Autoencoders (InfoVAE) family which is fast, easy to implement and leads to improvement in the unsupervised feature learning.

The main idea behind the MMD-VAE is to perform representation learning by encouraging a large mutual information between Z and X by using maximum mean discrepancy as a regularizer. MMD-VAE assumes that any two distributions are similiar if their moments are similiar. In frequency distribution, moments can be parameters of the distribution like mean, variance and skewness etc. It basically measures how the given two moments are different from each other and that can be accomplished by the following kernel embedding trick (Zhao et al., 2017). Here the kernel measures the similarity of two samples. If the kernel value is large, it means the two given samples are similar and if the value is small it can be interpreted as the samples are different.

$$\begin{aligned} D_{\mathrm{MMD}}(q\|p) = \mathbb{E}_{p(z),p(z')}\left[k\left(z,z'\right)\right] - 2\mathbb{E}_{q(z),p(z')}\left[k\left(z,z'\right)\right] \\ + \mathbb{E}_{q(z),q(z')}\left[k\left(z,z'\right)\right] \end{aligned} \tag{4.10}$$

where:

$k(z, z') =$ any universal kernel

$$k\left(z,z'\right) = e^{-\frac{\|z-z'\|^2}{2\sigma^2}}(Gaussian - kernel) \tag{4.11}$$

The final objective function of MMD-VAE is represented in the following equation. The first term is the reconstruction loss term and the second term is mmd loss term.

$$\mathcal{L}_{\mathrm{MMD-VAE}} = E_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] + D_{\mathrm{MMD}}\left(q_\phi(z)\|p_\theta(z)\right) \tag{4.12}$$

***β*-VAE**:

β-VAE is another variant of VAE which automatically find the disentangled factors in latent space. This is done by using the additional parameter β as D_{KL} weight. This constraint on KL divergence term allows the model to learn the independent latent factors effectively (Higgins et al., 2016). Thus the objective function becomes as follows:

$$\mathcal{L}(\theta,\phi,\beta) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \beta D_{KL}\left(q_\phi(z \mid x)\|p_\theta(z)\right) \tag{4.13}$$

When β is '1' then it is equivalent to standard VAE. One of the limitation of β-VAE is that it might ignore the latent information or leads to underfitting because of ineffective penalizing the weights of X and Z.

The above two variations of VAE are used in this research work to improve the existing vision (V) model in the traditional world model. The learning of disentangled representation has numerous advantages like good interpretability of latent space and also provides easy approximations to numerous tasks.

4.2.1.2 Advanced Memory Model

The architecture of world model clearly shows that the latent information which is encoded by the vision (V) model is given as input to the memory model to predict the next frame. This research believes that if the existing vision (V) model is improved then the latent information which is going as input to the memory (M) model also gets improved which implicitly improves the existing memory model in the traditional world model. Thus the improved vision (V) and memory (M) models increase the overall performance of world model by making it as the advanced world model.

Chapter 5

Experiments and Discussion of Results

This chapter discusses the results from the baseline and the improved approach for world models on CarRacing-v0 (Klimov, 2016) environment. Section 5.1 explains the dataset used in this research along with the preprocessing techniques. Section 5.2 presents the obtained results from the baseline and improved approaches. Section 5.3 compares the performance of baseline and improved architectures.

5.1 Dataset

OpenAI Gym is an opensource toolkit which provides numerous simulated environments for testing and developing reinforcement algorithms. This thesis work uses CarRacing-v0 (Klimov, 2016) environment for evaluating and comparing the performance of baseline and improved architectures. CarRacing-v0 is one of the simulated environments provided by openai gym which is a continuous control task. Section 5.1.1, Section 5.1.2 and Section 5.1.3 discusses in detail about the gaming environment, preprocessing techniques used to train the deep neural networks.

5.1.1 Dataset Exploration

CarRacing-v0 (Klimov, 2016) is a simple 2D racing RGB environment developed using Box2D (Catto, 2007) which is a 2D simulation library for games. It is a continuous control task which can be learned from pixels. The state of the environment consists of 96x96 pixels. The reward for each frame

is -0.1 and +1000/N for every track tile visited. Here N represents the total number of tiles through the entire track. An episode finishes when all the tiles are visited. With a track of 1000 frames, if the agent scores an average reward of 900 points consistently over 100 consecutive trails then it is considered as a successful game. In other case if the agent finish the game in 740 frames then the reward will be calculated as $1000 - 0.1 * 728 = 927.2$ points. Figure 5.1 gives a glimpse of whole environment. The red color car in the Figure 5.1 represents the agent, grey color path represents the track, red and white color stripes represents the borders along the track and the remaining area with green color represents the grass. Some other indicators like true speed, 4 ABS sensors steering wheel position and gyroscope are displayed at the bottom (black area) of the environment. If the agent is on the track then it will get the reward whereas when if it crosses the border and enters the grass area then it will get punishment 0f -100 and die.

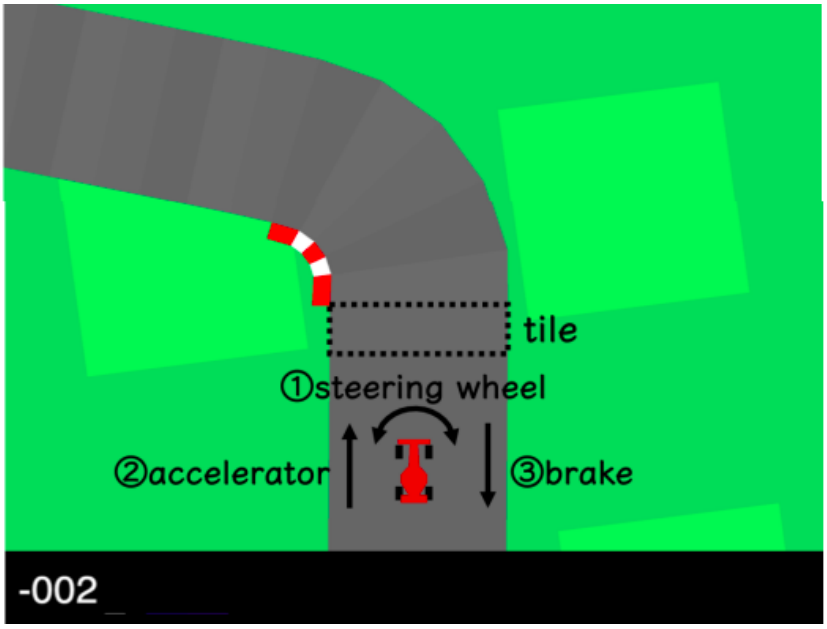


Figure 5.1: CarRacing-v0 environment
(Chang and Futagami, 2020; Klimov, 2016)

If we model the carracing environment using markov decision process (MDP), then it can be represented as follows:

- **Obseravtion space**: Each state consists of [STATE˙W $\times$ STATE˙H $\times$ channels] pixel information. So observation space is a set of $[96 \times 96 \times 3]$ vectors.

- **Action space**: It is a set of 3D vectors of the form $[s, a, b]$ where $\boldsymbol{s}$ represents the steering direction $(-1, 1)$, $\boldsymbol{a}$ represents acceleration $(0, 1)$ and $\boldsymbol{b}$ indicates brake $(0, 1)$

- **Reward**: Reward is -0.1 for every frame and +1000/N for every track tile visited, here N is the total number of tiles in track. For example, if the agent finishes the track in 728 frames, then reward is $1000 - 0.1 * 728 = 927.2$ points.

5.1.2 Dataset Preparation

This section discuses in detail about the preprocessing techniques used to prepare the data for training deep neural networks.

For this research, as a first step a total of 4000 rollouts/episodes are collected from the car racing environment using a random policy i.e observation data is generated by taking random actions and by exploring the environment multiple times. Each observation in the rollout is cropped to $[64 \times 64 \times 3]$ pixels to reduce the dimensions. Each rollout/episode is of 300 time-steps long and stored as a compressed (zip) numpy file which contains the observation data, action data, reward data and terminal state data in numpy format. In total, there will be $4000 \times 300 = 1200000$ observations/frames. Among these 4000 episodes, 3200 are used for training and 800 are used for testing the deep neural networks.

Experimental setting used for this research is different from the original baseline paper. Table 5.1 compares the important differences in experimental settings between this thesis work and baseline.

Table 5.1: Comparison of Experimental setting

	Baseline paper	**Thesis work**
Dataset	CarRacing-v0, VizDoom	CarRacing-v0
Size of observation	$64 \times 64 \times 3$ pixels	$64 \times 64 \times 3$ pixels
No.of rollouts	10000	4000
Population size (CMA-ES)	1024	4
No.of Generations	2000	50
No.of trainable parameters of vision (V) model	4,348,547	2,115,971
No.of trainable parameters of memory (M) model	422,368	423,649
No.of trainable parameters of controller (C) model	867	867

5.1.3 Training

The hyperparameter settings used for training all the models in this work are reviewed in Appendix B. Training procedure is divided into following steps:

- Step 1: As the training dataset is very large (3200 * 300 = 960000 observations) to fit in memory, first the 3200 episodes are divided into 16 batches where each batch contains 200 episodes (200 * 300 = 60000 observations).

- Step 2: Then the observation data [64 × 64 × 3] from the rollout data is given to vision (V) model which gives the compressed latent representation of original observation with the dimension of [32 × 32 × 3] pixels. The weights of the vision (V) model are saved in HDF5 format. Hierarchical Data Format (HDF5) (HDF-Group, 2020) allows to store huge amounts of numerical data and manipulate using numpy. Thus the vision (V) model is trained iteratively over 16 batches for 40 epochs and saved the vision (V) model weights using checkpoints.

- Step 3: As the world model architecture has different components and sequential pipeline for data transfer, another dataset is generated for training the memory (M) model. This memory (M) model uses teacher forcing method so the training dataset should be generated accordingly. The encoded/compressed observation along with the action data from the vision (V) model is given as input and one time-step ahead encoded/compressed observation is given as output to memory (M) model.

- Step 4: Using the training dataset generated for memory (M) model from step 3, memory (M) model is trained iteratively for 2000 time steps over the 16 batches and saved the memory (M) model weights using checkpoints.

- Step 5: Finally, the controller (C) model is trained using the saved weights from the vision (V) model and memory (M) model. As the controller (C) model uses evolution strategy to update the parameters of the model, the algorithm saves the best weights after every 25 generations which records the highest reward so far.

5.1.4 Loss functions

Loss functions are very important for evaluating the performance of algorithms. This section elaborates all the loss functions used in each compon-

ent of the world model.

Vision (V) model:

As we discussed earlier in Section 4.1.1.1, VAE is used in the vision (V) model and the loss of VAE is computed as the sum of reconstruction term and the kullback-leibler divergence term.

$$\mathcal{L}(\theta, \phi) = E_{q_\phi(z|x)} \left[\log p_\theta(x \mid z)\right] - D_{KL} \left(q_\phi(z \mid x) \| p_\theta(z)\right) \tag{5.1}$$

For the reconstruction term, Mean Squared Error (MSE) is used to compute the loss. In general, MSE is used in regression tasks. It computes the squared distance between the ground truth and predicted output. As we are dealing with 2D image frames in this research work, MSE calculates the squared distance between the original image and reconstructed image as follows:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \tag{5.2}$$

where,

y_i = original image
$\hat{y}_i$ = reconstructed image
N = Total number of dimensions

As the KL-divergence term computes the difference between the posterior and prior distributions, the closed form (Wei et al., 2020) of divergence is computed as follows:

$$D_{KL}[N(\mu(x), \Sigma(x)) \| N(0, 1)] \quad = \frac{1}{2} \sum_{k} \left((\exp \Sigma(x)) + \mu^2(x) - 1 - \log \Sigma(x)\right) \tag{5.3}$$

where,

$\mu(x), \Sigma(x)$ = Gaussian parameters

Advanced Vision model:

In this work, MMD-VAE and β-VAE are used in the advanced vision model. For MMD-VAE (Section 4.2.1.1), mean squared error is used for the

reconstruction term and Gaussian kernel has been used for the mean maximum discrepency term. The loss for β-VAE is computed similar to VAE loss but the only difference is the multiplication of hyperparameter β to the KL-divergence term.

Memory (M) model:

As the memory (M) model uses the MDN-RNN, the loss is computed by taking the negative logarithm of likelihood as follows:

$$E(y, \hat{y}) = -\ln \mathcal{L}(\hat{y} \mid x) = -\ln p(\hat{y} \mid x) \tag{5.4}$$

$$p(\hat{y} \mid x) = \frac{1}{\sqrt{2\sigma(x)^2\pi}} \exp\left(-\frac{(\hat{y} - \mu(x))^2}{2\sigma(x)^2}\right) \tag{5.5}$$

where:

$\mu(x)$ = Mean conditioned on input x
$\sigma(x)$ = Standard deviation conditioned on input x

A mixture of five Gaussian distributions are used in this research, so the final loss term is computed as the weighted sum of five Gaussian's of the negative logarithm of likelihood function.

Controller (C) model:

Controller (C) model uses an evolutionary algorithm called CMA-ES (Section 4.1.1.3), so it does not use any objective/loss function to optimize but instead it uses a simple linear model and finds the optimal parameters using the natural selection strategy.

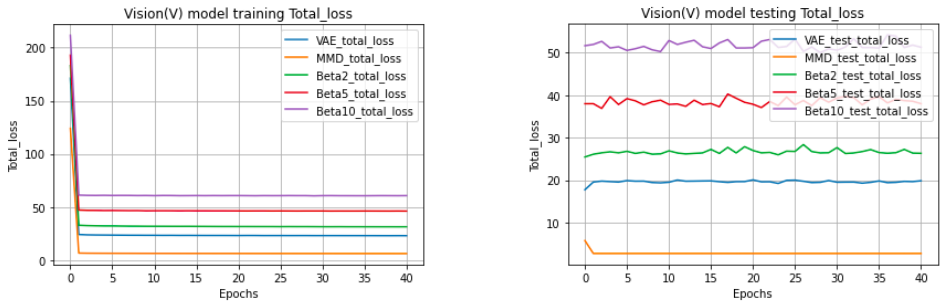
5.2 Experiments and Results

This section discusses the experiments which are conducted throughout this research work and clearly explains the results of each experiment. Section 5.2.1 explains the evaluation of vision (V) models, Section 5.2.2 describes the evaluation of memory (M) models and finally Section 5.2.3 examines the evaluation of controller (C) models.

5.2.1 Evaluation of vision (V) model

This section analyse the evaluation of vision models used in the baseline architecture as well as in the improved architecture. Variational AutoEncoder (VAE) has been used as a vision (V) model in the baseline architecture and Maximum Mean Discrepancy-VAE (MMD-VAE), β-VAE (with β=2,5,10) are used as the vision (V) models of improved architecture.

Figures 5.2, 5.3 and 5.4 depicts the total loss, divergence loss, reconstruction loss of different vision (V) models during training and testing phase. As we discussed earlier (Equation (5.1)), here total-loss is the sum of reconstruction-loss and the divergence-loss. For all the vision (V) models, reconstruction-loss is computed as the squared distance between the original pixel information and the reconstructed pixel information. The divergence-loss for VAE is the KL-divergence loss whereas for MMD-VAE it is the MMD-divergence loss and for β-VAE it is same as KL-divergence loss multiplied with corresponding β value. In this research, β values are taken as 2,5 and 10. All the vision (V) models are trained for 40 epochs.

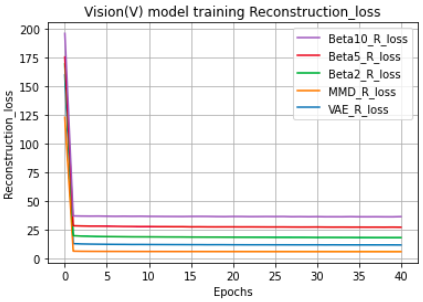


(a) Total-loss vs Epochs (Train)

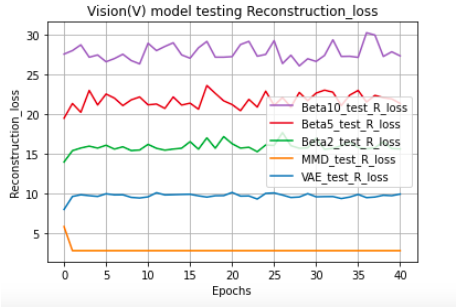
(b) Total-loss vs Epochs (Test)

Figure 5.2: Vision (V) model Total loss

This part elaborates the following learning curves of vision (V) models in detail. As we can see from the Figure 5.2, the total loss is decreasing for each of the vision (V) model during training. From the testing plot, it is clear that there is no overfitting as total loss for all the models is either decreasing or constant. Figure 5.3 shows that, other than β-VAE (β= 5,10) which has small peak in the begining of training of the divergence loss all the remaining models has decreasing behaviour in training and testing. Coming to the reconstruction loss from the Figure 5.4, learning curves shows the decreasing behaviour for all the models during training and shows some wobbly



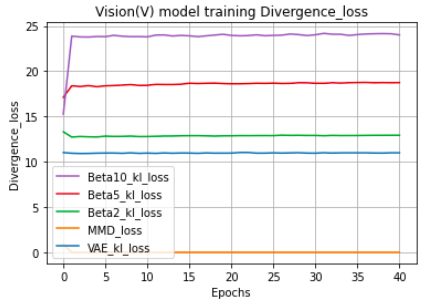
(a) Reconstruction-loss vs Epochs (Train)



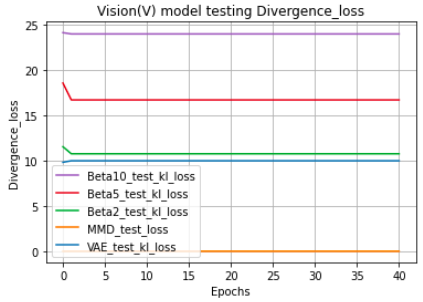
(b) Reconstruction-loss vs Epochs (Test)

Figure 5.4: Vision (V) model Reconstruction loss

behaviour during testing. Among all the vision (V) models used in this research, MMD-VAE is giving the minimum total loss. So it is considered for further research work to compare against the baseline architecture.



(a) Divergence-loss vs Epochs (Train)



(b) Divergence-loss vs Epochs (Test)

Figure 5.3: Vision (V) model Divergence loss

5.2.1.1 Reconstruction results from the vision (V) model

The following images depicts the reconstruction results from all the vision (V) models used in this research. One single frame from one episode is taken to compare the reconstructions against all the vision (V) models. This section discusses the important points from this reconstructed images from each of the vision (V) models.

The aim of successful reconstruction is to capture the important and useful features from the input image data. As we can see from Figure 5.5 the reconstruction of the VAE model is smooth but it was not able to capture the borders (red and white stripes) of the path which is the important part of the environment for the agent while taking right or left turns.

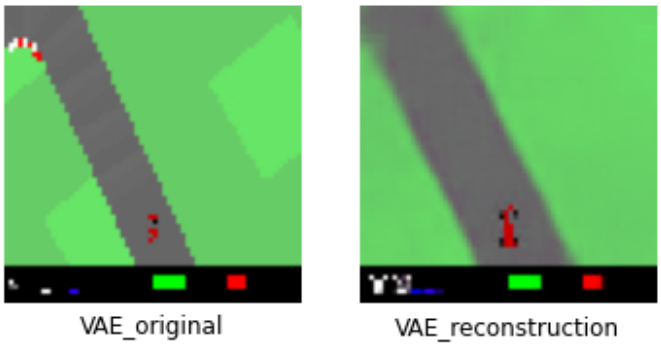


Figure 5.5: VAE original vs reconstruction (baseline)

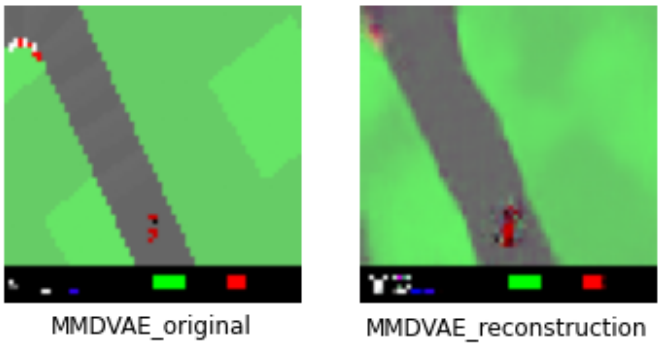


Figure 5.6: MMD-VAE original vs reconstruction

On the other-hand, MMDVAE which is a part of InfoVAE has the ability to capture the disentangled representations from the input image. Though the reconstructions from MMDVAE are blurry, it can be clearly seen from the Figure 5.6 that MMDVAE is able to capture the useful disentangled representations like it efficiently captured the borders of the path and also the patches of the green area. Eventhough the captured patches of the green grass area are not significant for this environment but it shows that how clearly the MMDVAE is able model separate dimensions of the input image. If we can add attention mechanism on top of captured useful and disentangled

features then it would be possible to yield better performance of the whole model. The Figures 5.7, 5.8, and 5.9 represent the reconstructions from the BetaVAE algorithm with $\beta = 2, 5 and 10$.

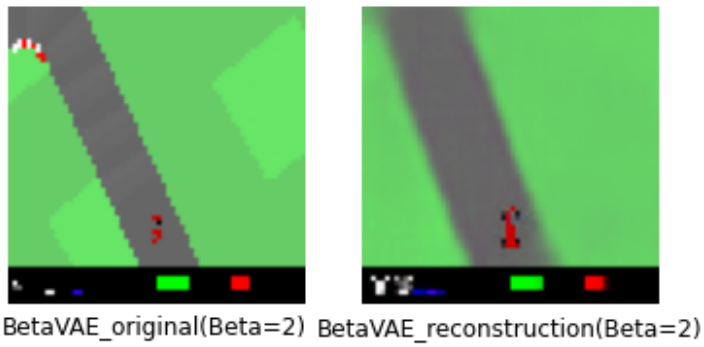


Figure 5.7: BetaVAE (β=2) original vs reconstruction

When $\beta = 1$ it is same as the standard VAE. By increasing the value of β, more weight is given to the kl-divergence term with the goal of learning disentangled representations. When we infer from the reconstructions of β-VAE model it is evident that the reconstructions are more blurred for the higher values of β.

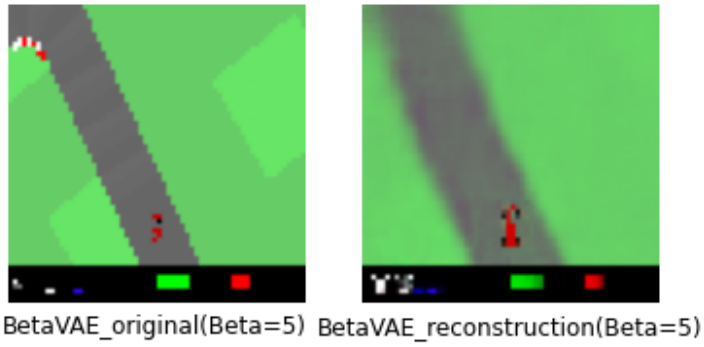


Figure 5.8: BetaVAE (β=5) original vs reconstruction

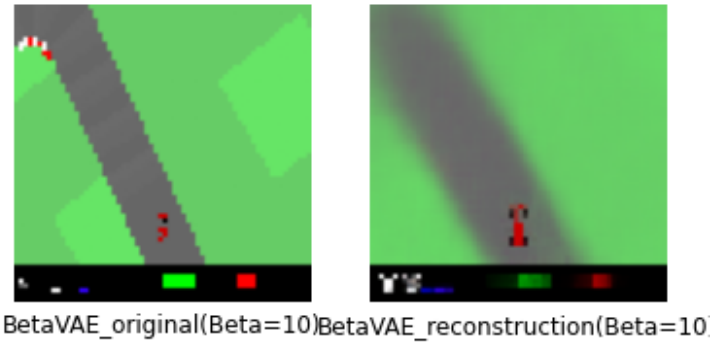


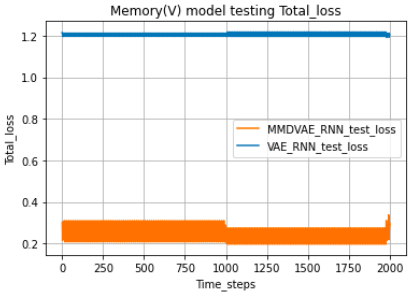
Figure 5.9: BetaVAE (β=10) original vs reconstruction

5.2.2 Evaluation of memory (M) model

This section discusses the evaluation of memory (M) model. In general, for training the memory (M) model a separate training dataset has to be generated. As per the baseline architecture, trained VAE model is used to generate one dataset and for the improved architecture, trained MMD-VAE model is used to generate another dataset for the memory (M) model. Then the memory (M) model is trained separately using these generated datasets and the performance is evaluated against both of them.

Figures 5.10, 5.11 and 5.12 depicts the total loss, prediction loss, reward loss of baseline and improved memory (M) models during training and testing phase. Here, total loss indicates the sum of prediction loss and reward loss. Prediction loss is the loss obtained when predicting the next future frame and it is calculated using Equation (5.5). Reward loss is computed using MSE i.e by taking the squared distance between the actual reward and the predicted reward.

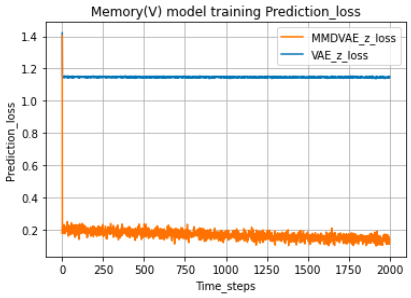
(a) Total-loss vs Time-steps (Train)



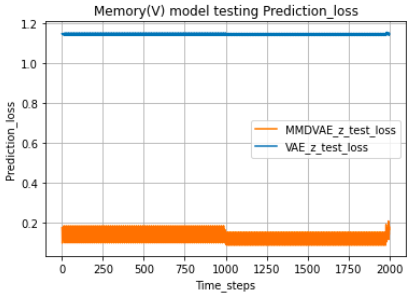
(b) Total-loss vs Time-steps (Test)

Figure 5.10: Memory (M) model Total loss

Let us discuss the learning curves of memory (M) model in detail. Figure 5.10 shows that the total loss is decreasing for both the baseline architecture and the improved architecture during training as well as in testing. Prediction loss and reward loss from Figures 5.11, 5.12 also shows the decreasing behaviour during both training and testing phase.



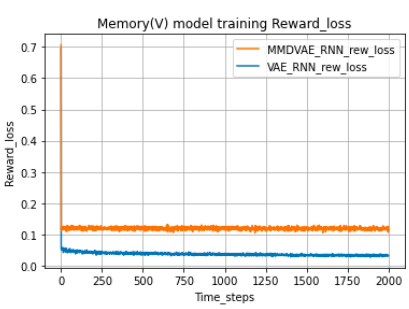
(a) Prediction-loss vs Time-steps (Train)



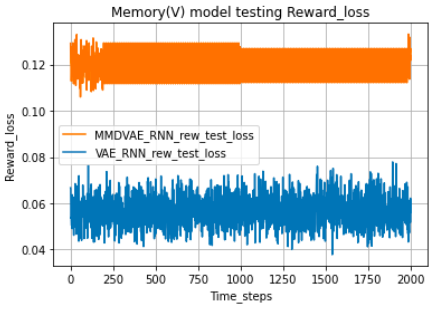
(b) Prediction-loss vs Time-steps (Test)

Figure 5.11: Memory (M) model Prediction loss

One important behaviour to notice here is that, for both the total loss and prediction loss improved memory (M) model gives the minimum loss whereas for reward loss, baseline memory (M) model is giving the minimum loss values.



(a) Reward-loss vs Time-steps (Train)

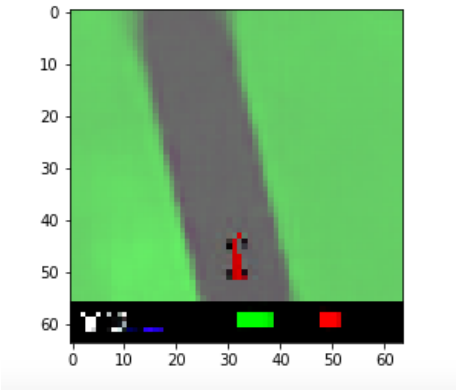


(b) Reward-loss vs Time-steps (Test)

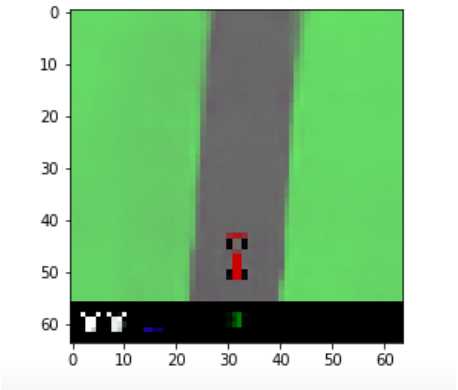
Figure 5.12: Memory (M) model Reward loss

5.2.2.1 Prediction results from the memory (M) model

Figure 5.13(a) represents the predicted image from the memory (M) model using the weights (VAE) from the baseline vision (V) model whereas the Figure 5.13(b) represents the predicted image from the memory (M) model using the weights (MMDVAE) from the improved vision (V) model. Though there is no much difference between the two predictions, we can infer from the learning curves that the total-loss of the memory (M) model which uses the weights from the improved vision (V) model gives the minimum error rate.



(a) Baseline model prediction



(b) Improved model prediction

Figure 5.13: Prediction of next frame by Memory (M) model

5.2.3 Evaluation of controller (C) model

Training and evaluation of controller (C) model will be explained in this section. As discussed earlier, controller (C) model is trained using an evolution strategy called CMA-ES.

Controller (C) model accepts the input vector of dimension 32 (V) + 256 (M) = 288 and outputs a vector of dimension 3 (steering angle, acceleration and brake). So in total, it has $288 \times 3 + 1(bias) = 867$ parameters to train. In order to optimize the 867 trainable parameters, CMA-ES first creates the multiple copies (population size) of 867 parameters with random initialization. Due to resource constraints, population (p) of size 2 and 4 are used in this research work. Total population size is computed by multiplying the number of workers (system cores) with number of members of population given to each worker for testing. Then the model tests each member of the population and records average reward. Finally, the weights which corresponds to highest reward will be used to produce the next generation until the parameters are optimized. For this research, controller (C) model is trained for 50 generations.

At the end, best weights from the trained controller (C) model are used to run over a 100 episodic trails to test the performance of the model. Figure 5.14 depicts the average reward achieved by an agent over 100 trails of episodes. The blue and green curves indicates the average reward obtained by using the models in the baseline architecture (VAE-MDNRNN) with population size of $2, 4$ and the red and orange curves indicates the average reward obtained from the models of the improved architecture (MMD-MDNRNN) with population size of $2, 4$.

5.3 Baseline vs Improved Approach

This section compares the performance of improved architecture against the baseline architecture. The final performance is compared based on the final average reward obtained by the agent over the 100 episodic trails. Table 5.2 represents the average error rate obtained by the vision (V), memory (M) models and average reward obtained by the controller (C) model using baseline and improved architectures.

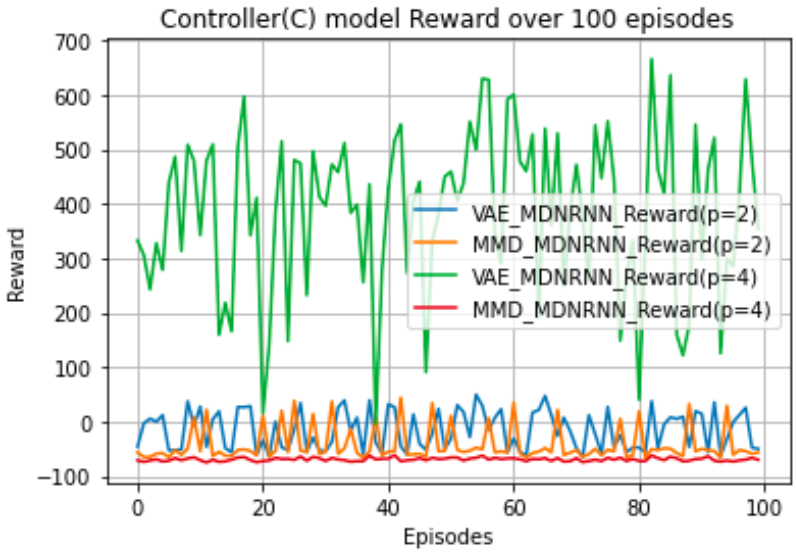


Figure 5.14: Controller (C) model Reward over 100 trails

Architecture	Baseline	Improved
Vision (V) model	11.9019	6.0441
Memory (M) model	1.1463	0.1155
Controller (C) model	-16/+386	-49/-69

The table represents the comparison between the baseline and improved model. The results from the vision (V), memory (M) model indicates the average loss values whereas the results from the controller (C) model indicates the average reward values with population size of 2, 4.

Table 5.2: Baseline vs Improved Results

From the above Table 5.2, we can infer that the improved architecture is giving the minimum error rate for both the vision (V) and memory (M) models whereas the baseline architecture gives the maximum average reward for the controller (C) model. Green color results represents the corresponding minimum error rates for the vision (V), memory (M) models and maximum average reward for the controller (C) model.

5.4 Discussion

This section presents the interpretation and discussion on the obtained results from the Table 5.2.

As we discussed before, vision (V) model use the deep generative models to learn the spatial representation of input data. The encoder generates the low-dimensional latent representation from which the decoder reconstructs the original input. So, it is inappropriate to compute the accuracy of this vision (V) model because the reconstructions are done from the latent representations. Hence, the average loss is taken as the metric to compare the performance of various vision (V) models used in this work. Among all the vision (V) models used in this research, MMD-VAE gave the minimum average loss (6.0441). This occurs due to the nature of MMD-VAE which always tries to match the prior distribution whereas the standard VAE will be very far from true prior. It is also evident from the obtained reconstructed images that MMD-VAE is able to capture important features of the input image.

The memory (M) model that predicts the future images by taking the latent information generated from the MMD-VAE gave the minimum average error (0.1155) compared to the baseline (1.1463). Again, the average loss is considered as a metric to compare the performance of memory (M) models because the prediction of future images are done based on the latent representations given by the vision (V) model.

Due to limited time and resource bounds, controller (C) model is trained for very few generations (50) with a small population size of $(2, 4)$. With the population size of 2, controller (C) model gave the negative average reward for both the baseline (-16) and improved (-49) models but with the population size of 4, it gave the positive average reward for baseline ($+386$) and negative average reward for improved (-69) model. Thus the better performance is given by the baseline controller (C) model. The reason behind the behavior of controller (C) model is due to the lack of sufficient training of the controller (C) model or due to inability of controller (C) model to understand the disentangled representations generated by improved vision (V) and memory (M) models.

Overall, the vision (V) and memory (M) models of the advanced world model showed the better performance by giving the less average error and also by generating the more informative latent representations. However, the controller (C) model of the world model gave the highest average reward.

Chapter 6

Conclusion

This thesis work aims to improve the existing world models by using the advanced variations of deep generative neural networks. It is believed that improvement in atleast one component of the whole world model leads to overall performance improvement of the deep RL agent in the world model to solve the given task. This research work has successfully improved the vision (V) model by using a MMDVAE which is a family of InfoVAE instead of standard VAE. The latent information generated from the improved vision (V) model is made disentangled and informative. The improvement in the vision (V) model leads to the implicit improvement of memory (M) model. Even though both the vision (V) and memory (M) models are improved, the controller which receives the input from both of them has not improved due to lack of enough training. Thus this thesis work focused on the improvement of vision (V) model that leads to the generation of high quality simulation data. The informative simulation data can be helpful for the RL agent to learn the task better. The experiments conducted in this research shows that the improved model is able to capture the disentangled features of the input data and also important features to solve the given task. The stated research questions in this thesis are answered as follows:

1. *Does this research help to improve the performance of existing traditional world model?*

   Yes, as stated in the motivation this research has improved the performance of vision (V) and memory (M) model of baseline by getting the minimum average loss when compared to the traditional world model. But the average reward of the controller (C) model has not improved when compared to the baseline which has to further investigated in future works.

2. *How good is the advanced vision model compared to the existing vision model?*

   MMD-VAE, β-VAE has been used as vision (V) models in improved model. From the results, we can infer that the MMD-VAE outperformed the standard VAE by giving the minimum average error and also disentangled representations of given data.

3. *How are the results of the improved model evaluated?*

   To evaluate the performance of the improved model, the results are compared against the baseline model. The average loss is chosen as a metric for both the vision (V) and memory (M) models whereas for the controller (C) model, average reward has been chosen to compare the results against baseline and improved model.

6.1 Future Works

In future research, temporal convolutional networks (TCN) and transformers can be used instead of memory (M) model to improve the quality of simulation data that helps to improve the overall performance of world models. Temporal convolutional networks (TCN) are good at dealing with long range dependencies and transformers uses attention mechanism that focus on important parts of the input. TCN's and transformers also support parallelization which can significantly decrease the time complexity.

Literature (Robine et al., 2020) has proved that controller (C) model has shown better performance by employing gradient based techniques for optimizing the parameters of the model. Thus, more research has to be performed on controller (C) model in future works.

Bibliography

Atkeson, Christopher G and Juan Carlos Santamaria (1997). "A comparison of direct and model-based reinforcement learning". In: *Proceedings of international conference on robotics and automation*. Vol. 4. IEEE, pp. 3557–3564.

Bai, Shaojie, J Zico Kolter and Vladlen Koltun (2018). "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling". In: *arXiv preprint arXiv:1803.01271*.

Bakker, Bram et al. (2006). "Quasi-online reinforcement learning for robots". In: *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.* IEEE, pp. 2997–3002.

Bishop, Christopher M (1994). "Mixture density networks". In:

Bre, Facundo, Juan M Gimenez and Víctor D Fachinotti (2018). "Prediction of wind pressure coefficients on building surfaces using artificial neural networks". In: *Energy and Buildings* 158, pp. 1429–1441.

Burgess, Christopher P et al. (2018). "Understanding disentangling in β-VAE". In: *arXiv preprint arXiv:1804.03599*.

Camuñas-Mesa, Luis A, Bernabé Linares-Barranco and Teresa Serrano-Gotarredona (2019). "Neuromorphic spiking neural networks and their memristor-CMOS hardware implementations". In: *Materials* 12.17, p. 2745.

Catto, Erin (2007). *Box2D*. URL: `https://box2d.org/documentation/index.html` (visited on 20/10/2021).

Chadha, Parth and Deepika Bablani (2018). "Incorporating Attention in World Models for Improved Dynamics Modeling". In:

Chang, Hanten and Katsuya Futagami (2020). "Reinforcement learning with convolutional reservoir computing". In: *Applied Intelligence* 50.8, pp. 2400–2410.

Chung, Junyoung et al. (2015). "Gated feedback recurrent neural networks". In: *International conference on machine learning*. PMLR, pp. 2067–2075.

Depraetere, B et al. (2014). "Comparison of model-free and model-based methods for time optimal hit control of a badminton robot". In: *Mechatronics* 24.8, pp. 1021–1030.

Doersch, Carl (2021). *Tutorial on Variational Autoencoders*. arXiv: `1606.05908 [stat.ML]`.

Doya, Kenji (1993). "Bifurcations of recurrent neural networks in gradient descent learning". In: *IEEE Transactions on neural networks* 1.75, p. 218.

Feng, Junxi et al. (2019). "Reconstruction of porous media from extremely limited information using conditional generative adversarial networks". In: *Physical Review E* 100.3, p. 033308.

Freeman, C Daniel, Luke Metz and David Ha (2019). "Learning to predict without looking ahead: World models without forward prediction". In: *arXiv preprint arXiv:1910.13038*.

Frydenberg, Morten (1990). "The chain graph Markov property". In: *Scandinavian Journal of Statistics*, pp. 333–353.

Goodfellow, Ian, Yoshua Bengio and Aaron Courville (2016). *Deep learning*. MIT press.

Graves, Alex (2013). "Generating sequences with recurrent neural networks". In: *arXiv preprint arXiv:1308.0850*.

Ha, David and Jürgen Schmidhuber (2018). "World models". In: *arXiv preprint arXiv:1803.10122*.

Hafner, Danijar et al. (2019). "Dream to control: Learning behaviors by latent imagination". In: *arXiv preprint arXiv:1912.01603*.

Hafner, Danijar et al. (2020). "Mastering atari with discrete world models". In: *arXiv preprint arXiv:2010.02193*.

Hansen, Nikolaus (2006). "The CMA evolution strategy: a comparing review". In: *Towards a new evolutionary computation*, pp. 75–102.

Hansen, Nikolaus and Andreas Ostermeier (2001). "Completely derandomized self-adaptation in evolution strategies". In: *Evolutionary computation* 9.2, pp. 159–195.

HDF-Group (2020). *HDF5 for python*. URL: https://docs.h5py.org/en/stable/build.html (visited on 18/08/2021).

Henderson, Peter et al. (2018a). "Deep reinforcement learning that matters". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1.

Henderson, Peter et al. (2018b). "Deep reinforcement learning that matters". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1.

Higgins, Irina et al. (2016). "beta-vae: Learning basic visual concepts with a constrained variational framework". In:

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long short-term memory". In: *Neural computation* 9.8, pp. 1735–1780.

JORDAN, JEREMY (2018). *Setting the learning rate of your neural network.* URL: https://www.jeremyjordan.me/nn-learning-rate/ (visited on 04/10/2021).

Kingma, Diederik P and Jimmy Ba (2014). "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980*.

Kingma, Diederik P and Max Welling (2013). "Auto-encoding variational bayes". In: *arXiv preprint arXiv:1312.6114*.

Klimov, Oleg (2016). *Carracing-v0*. URL: https://gym.openai.com/envs/CarRacing-v0/.

Kubat, Miroslav (2017). *An introduction to machine learning*. Springer.

Lange, Robert Tjarko (2020). *Getting started with JAX (MLPs, CNNs and RNNs)*. URL: https://roberttlange.github.io/posts/2020/03/blog-post-10/ (visited on 20/10/2021).

Lateef, Zulaikha (2021). *Types Of Artificial Intelligence You Should Know*. URL: https://www.edureka.co/blog/types-of-artificial-intelligence/ (visited on 04/10/2021).

Liang, Jacky et al. (2018a). "GPU-accelerated robotic simulation for distributed reinforcement learning". In: *Conference on Robot Learning*. PMLR, pp. 270–282.

Liang, Xingxing et al. (2018b). "VMAV-C: A deep attention-based reinforcement learning algorithm for model-based control". In: *arXiv preprint arXiv:1812.09968*.

Majid, Amjad Yousef et al. (2021). "Deep Reinforcement Learning Versus Evolution Strategies: A Comparative Survey". In: *arXiv preprint arXiv:2110.01411*.

Mao, Hongzi et al. (2016). "Resource management with deep reinforcement learning". In: *Proceedings of the 15th ACM workshop on hot topics in networks*, pp. 50–56.

Mathieu, Emile et al. (2019). "Disentangling disentanglement in variational autoencoders". In: *International Conference on Machine Learning*. PMLR, pp. 4402–4412.

McCarthy, John (2007). "What is artificial intelligence?" In:

Pai, Aravind (2020). *CNN vs. RNN vs. ANN – Analyzing 3 Types of Neural Networks in Deep Learning*. URL: `https://www.analyticsvidhya.com/blog/2020/02/cnn-vs-rnn-vs-mlp-analyzing-3-types-of-neural-networks-in-deep-learning/` (visited on 05/10/2021).

Park, Hwin Dol, Youngwoong Han and Jae Hun Choi (2019). "Medical Time-series Prediction With LSTM-MDN-ATTN". In: *2019 International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, pp. 1359–1361.

Plesiak, Marianna (2020). *Recurrent neural networks and their applications in NLP*. URL: `https://compstat-lmu.github.io/seminar_nlp_ss20/recurrent-neural-networks-and-their-applications-in-nlp.html` (visited on 09/10/2021).

Polydoros, Athanasios S and Lazaros Nalpantidis (2017). "Survey of model-based reinforcement learning: Applications on robotics". In: *Journal of Intelligent & Robotic Systems* 86.2, pp. 153–173.

Pu, Yunchen et al. (2016). "Variational autoencoder for deep learning of images, labels and captions". In: *Advances in neural information processing systems* 29, pp. 2352–2360.

Rezende, Danilo Jimenez, Shakir Mohamed and Daan Wierstra (2014). "Stochastic backpropagation and approximate inference in deep generative models". In: *International conference on machine learning*. PMLR, pp. 1278–1286.

Risi, Sebastian and Kenneth O Stanley (2019). "Deep neuroevolution of recurrent and discrete world models". In: *Proceedings of the Genetic and Evolutionary Computation Conference*, pp. 456–462.

Robine, Jan, Tobias Uelwer and Stefan Harmeling (2020). "Smaller World Models for Reinforcement Learning". In: *arXiv preprint arXiv:2010.05767*.

Salimans, Tim et al. (2017). "Evolution strategies as a scalable alternative to reinforcement learning". In: *arXiv preprint arXiv:1703.03864*.

Schatzmann, Jost et al. (2006). "A survey of statistical user simulation techniques for reinforcement-learning of dialogue management strategies". In: *The knowledge engineering review* 21.2, pp. 97–126.

Schuster, Mike and Kuldip K Paliwal (1997). "Bidirectional recurrent neural networks". In: *IEEE transactions on Signal Processing* 45.11, pp. 2673–2681.

Sejnowski, Terrence J (2020). "The unreasonable effectiveness of deep learning in artificial intelligence". In: *Proceedings of the National Academy of Sciences* 117.48, pp. 30033–30038.

SiriTeam (2017). *Deep Learning for Siri's Voice: On-device Deep Mixture Density Networks for Hybrid Unit Selection Synthesis*. URL: `https://machinelearning.apple.com/research/siri-voices` (visited on 10/10/2021).

Sivamani, Kirthi Shankar (2019). *Back-Propagation simplified*. URL: `https://towardsdatascience.com/back-propagation-simplified-218430e21ad0` (visited on 04/10/2021).

Sphinx (2018). *Key Concepts in RL*. URL: `https://spinningup.openai.com/en/latest/spinningup/rl_intro.html#id2` (visited on 05/10/2021).

Sra, Suvrit, Sebastian Nowozin and Stephen J Wright (2012). *Optimization for machine learning*. Mit Press.

Swapna (2021). *Convolution Neural Networks*. URL: `https://developersbreach.com/convolution-neural-network-deep-learning/` (visited on 08/10/2021).

Tan, U et al. (2019). "On the Eclipsing Phenomenon with Phase Codes". In: *2019 International Radar Conference (RADAR)*. IEEE, pp. 1–5.

Tang, Yujin, Duong Nguyen and David Ha (2020). "Neuroevolution of self-interpretable agents". In: *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*, pp. 414–424.

Varsamopoulos, Savvas, Koen Bertels and Carmen Almudever (2018). *Designing neural network based decoders for surface codes*. Nov. 2018.

Vossen, Julian, Baptiste Feron and Antonello Monti (2018). "Probabilistic forecasting of household electrical load using artificial neural networks".

In: *2018 IEEE International Conference on Probabilistic Methods Applied to Power Systems (PMAPS)*. IEEE, pp. 1–6.

Wei, Ruoqi et al. (2020). "Variations in variational autoencoders-a comparative evaluation". In: *Ieee Access* 8, pp. 153651–153670.

Weng, Lilian (2018). *From Autoencoder to Beta-VAE*. URL: https://lilianweng.github.io/lil-log/2018/08/12/from-autoencoder-to-beta-vae.html (visited on 15/10/2021).

Williams, Ronald J and David Zipser (1989). "A learning algorithm for continually running fully recurrent neural networks". In: *Neural computation* 1.2, pp. 270–280.

Yamashita, Rikiya et al. (2018). "Convolutional neural networks: an overview and application in radiology". In: *Insights into imaging* 9.4, pp. 611–629.

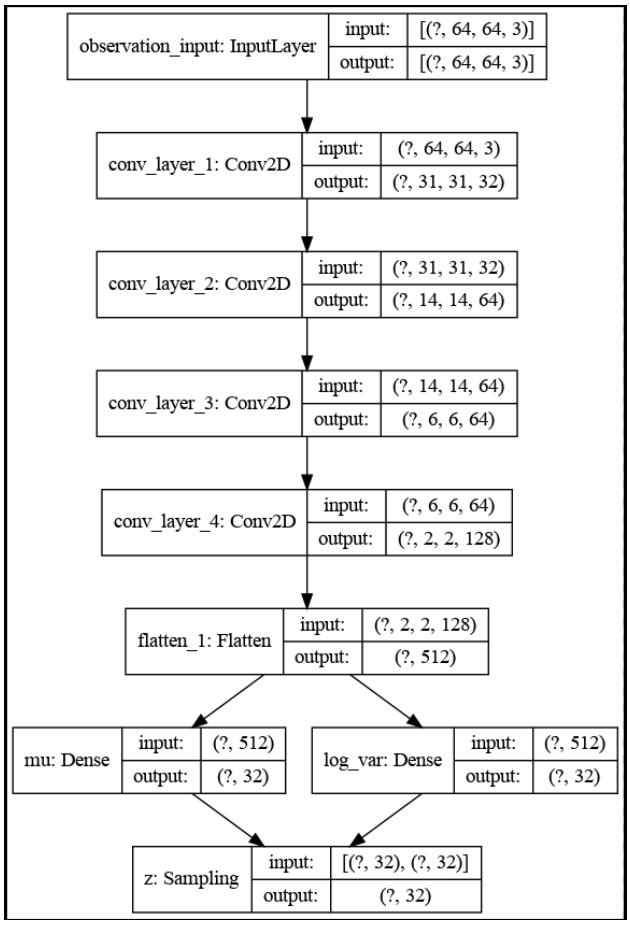
Zhao, Shengjia, Jiaming Song and Stefano Ermon (2017). "Infovae: Information maximizing variational autoencoders". In: *arXiv preprint arXiv:1706.02262*.

Appendices

Appendix A

Model Architectures

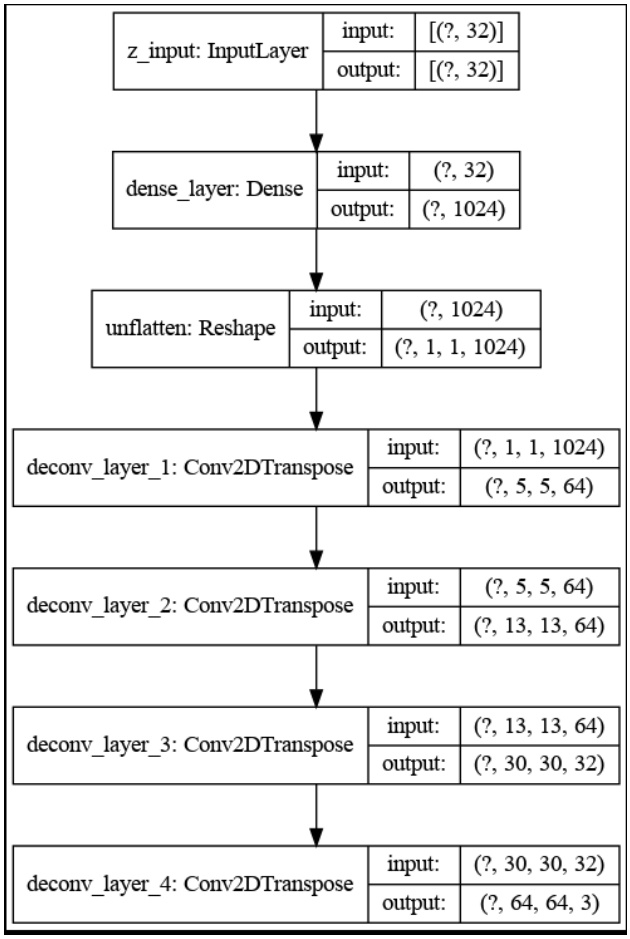
A.1 Encoder architecture of vision (V) model



A.1.1 Trainable parameters of encoder model

```
Model: "encoder"
__________________________________________________________________________________________________
Layer (type)                    Output Shape         Param #     Connected to
==================================================================================================
observation_input (InputLayer)  [(None, 64, 64, 3)]  0
__________________________________________________________________________________________________
conv_layer_1 (Conv2D)           (None, 31, 31, 32)   1568        observation_input[0][0]
__________________________________________________________________________________________________
conv_layer_2 (Conv2D)           (None, 14, 14, 64)   32832       conv_layer_1[0][0]
__________________________________________________________________________________________________
conv_layer_3 (Conv2D)           (None, 6, 6, 64)     65600       conv_layer_2[0][0]
__________________________________________________________________________________________________
conv_layer_4 (Conv2D)           (None, 2, 2, 128)    131200      conv_layer_3[0][0]
__________________________________________________________________________________________________
flatten_2 (Flatten)             (None, 512)          0           conv_layer_4[0][0]
__________________________________________________________________________________________________
mu (Dense)                      (None, 32)           16416       flatten_2[0][0]
__________________________________________________________________________________________________
log_var (Dense)                 (None, 32)           16416       flatten_2[0][0]
__________________________________________________________________________________________________
z (Sampling)                    (None, 32)           0           mu[0][0]
                                                                 log_var[0][0]
==================================================================================================
Total params: 264,032
Trainable params: 264,032
Non-trainable params: 0
```

A.2 Decoder architecture of vision (V) model

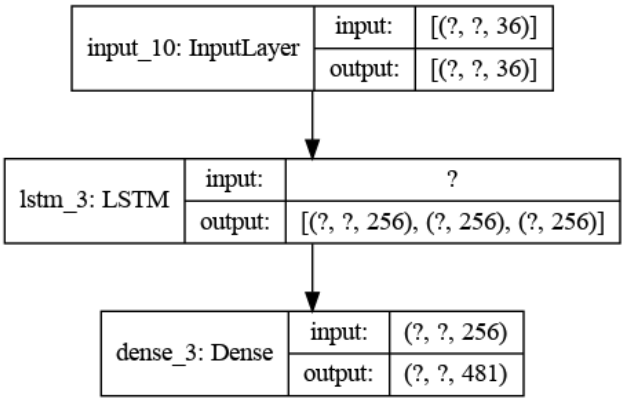


A.2.1 Trainable parameters of decoder model

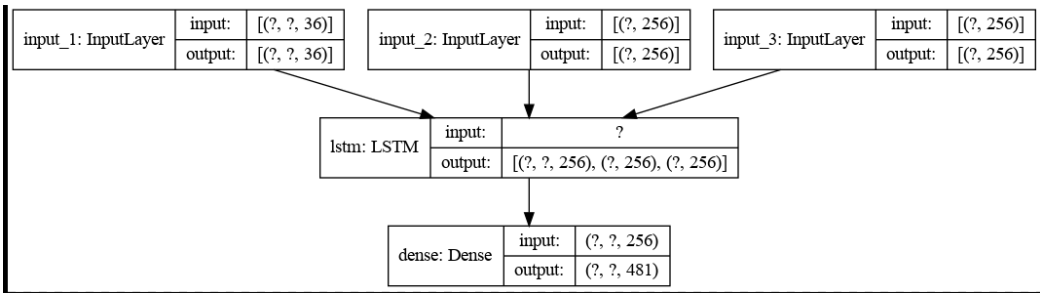
```
Model: "decoder"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
z_input (InputLayer)         [(None, 32)]              0
_________________________________________________________________
dense_layer (Dense)          (None, 1024)              33792
_________________________________________________________________
unflatten (Reshape)          (None, 1, 1, 1024)        0
_________________________________________________________________
deconv_layer_1 (Conv2DTransp (None, 5, 5, 64)          1638464
_________________________________________________________________
deconv_layer_2 (Conv2DTransp (None, 13, 13, 64)        102464
_________________________________________________________________
deconv_layer_3 (Conv2DTransp (None, 30, 30, 32)        73760
_________________________________________________________________
deconv_layer_4 (Conv2DTransp (None, 64, 64, 3)         3459
=================================================================
Total params: 1,851,939
Trainable params: 1,851,939
Non-trainable params: 0
```

A.3 MDN-RNN architecture

A.3.1 Memory (M) model during training



A.3.2 Memory (M) model during inference



A.3.3 Trainable parameters of memory (M) model

```
Model: "model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         [(None, None, 36)]        0
_________________________________________________________________
lstm (LSTM)                  [(None, None, 256), (None 300032
_________________________________________________________________
dense (Dense)                (None, None, 481)         123617
=================================================================
Total params: 423,649
Trainable params: 423,649
Non-trainable params: 0
```

Appendix B

Hyperparameter Settings

Hyperparameter	**Value**
Latent dimensions	32
Batch size	100
Learning rate	0.0001
Epochs	40
β Values	2, 5, 10

Table B.1: Hyperparameter setting for vision (V) model

Hyperparameter	Value
Batch size	100
Learning rate	0.00001
Time-steps	2000
LSTM hidden units	256
Gaussian mixtures	5

Table B.2: Hyperparameter setting for memory (M) model

Hyperparameter	Value
Population size	2, 4
Number of workers	2
Number of trials per worker	1, 2
Number of generations used to evaluate performance	50

Table B.3: Hyperparameter setting for Controller (C) model
Population size = Number of workers $*$ Number of trials per worker